

# Example-Driven Intent Synthesis for Constrained Data Bundle Retrieval: Focused Text Snippet Extraction and Beyond

Whanhee Cho  
University of Utah  
whanhee.cho@utah.edu

Kuangfei Long  
Boston University  
longkuangfei@outlook.com

Mahmood Jasim  
Louisiana State University  
mjasim@lsu.edu

Matteo Brucato  
OSM Data  
matteo@osm-data.com

Alexandra Meliou  
UMass Amherst  
ameli@cs.umass.edu

Peter J. Haas  
UMass Amherst  
phaas@cs.umass.edu

Anna Fariha  
University of Utah  
afariha@cs.utah.edu

## ABSTRACT

Selecting a *bundle* of items that collectively satisfies constraints is a fundamental task across databases, recommender systems, and text summarization. Unlike traditional retrieval that returns individual or top- $k$  items, bundle retrieval is inherently combinatorial and, in general, NP-hard. Although package queries can efficiently retrieve bundles given a well-formed query, two key user-centric challenges remain: (1) expressing and tuning multi-dimensional bundle intent through a user-friendly interface, and (2) ensuring feasibility when the query yields empty results. We introduce Ex2BUNDLE, an *Example-driven Bundle retrieval* framework that enables users to specify their intent through *example bundles* and automatically synthesizes package queries that capture the intent implicit in those example bundles via aggregate constraints. Ex2BUNDLE also addresses a challenge unique to bundle retrieval: when inferred aggregate constraints are infeasible over the target data, our *data-aware constraint relaxation* minimally adjusts the constraint bounds while preserving alignment with user intent. We instantiate a specific application of *focused text snippet extraction by example* to demonstrate the efficacy of the Ex2BUNDLE framework. Extensive experiments over real-world datasets and a user study demonstrate that Ex2BUNDLE improves usability and consistently returns intent-aligned bundles even under distributional shifts of the target database.

**Artifact Link:** <https://github.com/kuangfei-long/ex2bundle>.

## 1 INTRODUCTION

Data *bundle* retrieval is a fundamental task across multiple domains: package queries retrieve packages (sets of tuples) from relational databases under aggregate constraints [11]; recommender systems suggest playlists (sets of songs) or combo offers (sets of products) tailored to user preferences [4]; and extractive text summarization systems retrieve subsets of sentences from documents that form coherent summaries based on user queries [21, 58, 81]. Unlike traditional retrieval that returns individual or top- $k$  items, bundle retrieval involves selecting a *set* (or package [11]) whose elements must collectively optimize global objectives while satisfying set-level constraints that align with user preferences. Consequently, the inclusion or exclusion of any single item affects the feasibility and overall quality of the bundle, making the problem inherently combinatorial and NP-hard [11]. While systems exist for the efficient retrieval of data bundles from this combinatorial search space [11], two user-centric challenges remain as primary obstacles: (1) the *interface* challenge—how can humans easily express their intent of

the desired data bundles—and (2) the *feasibility* challenge—how to ensure non-empty answers for the user queries.

**Challenge 1: Usable interface.** The first challenge is communicating the user intent of the desired data bundle. Like SQL, the Package Query Language (PaQL) [11] allows specification of a bundle query through linear constraints and objective functions over data attributes, but the users must know the exact query parameters: attribute names and their bounds as numerical values. This is inherently difficult for human users—especially non-experts—who must (i) learn the PaQL [11] syntax, (ii) know the database schema, (iii) determine the attributes of interest, (iv) have a good sense of the value distributions, and (v) translate their intent precisely to parameterized constraints and global objectives of a PaQL query. We show in Example 1 that even for experts, step (v) is particularly difficult for PaQL because constraints are over *bundle aggregates* (SUM, AVG, COUNT): users must reason at the bundle level—predicting what a SUM should be over a bundle of unspecified size—rather than at the tuple level as in SQL. For certain applications, such as extractive document summarization [81], an alternative is to specify the intent in natural language. However, as prior work shows, natural language is too generic to express specific user intents [21] and subsequent conversational tuning of intent is imprecise and frustrating for humans [84].

**EXAMPLE 1 (SYNTHESIZING A PACKAGE QUERY).** *Morpheus, a CS department Chair, is seeking to fill two tenure-track assistant professor positions to strengthen AI and Databases. Since he plans for the two new hires to jointly teach an “AI in Databases” course, he wants them to have reasonable collective teaching experience, but not too much, as that may indicate teaching-focused or senior candidates who are unlikely to accept a tenure-track entry-level position. He also wants to maximize the total recommendation score of the new hires based on their letters. This is a bundle retrieval problem: Morpheus must pick a set of two out of 200 candidates that best satisfy his criteria, as shown via the following under-specified package query:*<sup>1</sup>

```
Q1:  SELECT PACKAGE(*) FROM CANDIDATES
      SUCH THAT COUNT(*) = 2
      AND SUM(ai_score) is high
      AND SUM(db_score) is high
      AND SUM(teaching_score) is reasonable
      MAXIMIZE SUM(reco_score);
```

<sup>1</sup>For ease of exposition, we use a toy example, albeit somewhat unrealistic, of 2 hires over 4 simple criteria; in practice, faculty searches involve more candidates (5–10) over more complex criteria (15+). The search space grows combinatorially with the #candidates. E.g., choosing 5 out of 200 yields  $\binom{200}{5} \approx 2.5$  billion possible bundles.

|       | AI       | Databases | Teaching       | Recommendation |
|-------|----------|-----------|----------------|----------------|
|       | ai_score | db_score  | teaching_score | reco_score     |
| Smith | 0.8      | 0.4       | 0.1            | 0.4            |
| Jones | 0.4      | 0.7       | 0.2            | 0.9            |
| Neo   | 0.4      | 0.5       | 0.4            | 0.8            |
| Brown | 0.5      | 0.4       | 0.4            | 0.9            |
| ...   | ...      | ...       | ...            | ...            |

Table 1: Partial list of candidates for Example 1.

However, at this point, all Morpheus has are the raw application materials (resumes, statements, letters, etc.) of these candidates. Before querying the candidate database, he must first map each candidate’s application materials to a 4-dimensional vector space: `ai_score`, `db_score`, `teaching_score`, and `reco_score`. Morpheus decides to use an off-the-shelf embedding technique to obtain numeric scores for each of these dimensions for every candidate (Table 1). However, he now faces another problem: he does not know what constitutes an appropriate replacement for the underspecified values *high* and *reasonable* in Q1. Is 0.7 considered a *high* score for AI expertise? What value represents the *reasonable* range for teaching score? He can examine the value distributions of the embeddings, but still cannot determine which corresponding values would accurately capture his intended criteria.

Example 1 highlights two key struggles in formulating package queries to express bundle query intent: (1) the lack of means to accurately map a database to a vector database with appropriate dimensions of interest, which primarily affects non-experts; and (2) the lack of knowledge of the correct parameters for the PaQL query, which affects experts and non-experts alike. In Example 1, Morpheus was interested in only 4 criteria; however, his struggle would be even worse if there were a larger number of criteria.

**Our solution to Challenge 1: bundle-query by example.** To overcome the usability challenge in the interface of bundle query intent specification, we build on the *Query by Example* (QbE) paradigm [88], which has seen significant success in traditional SQL querying [23] and other domains [21, 29, 30, 62] by lowering the barrier to task specification. We extend QbE to bundle queries (BQbE): the user provides a few exemplar data bundles to convey their query intents *implicitly*, thereby bypassing the construction of a precisely parameterized PaQL query. BQbE is particularly useful when users cannot articulate precise criteria but can instead provide representative examples of what they seek. For a large number of use cases, BQbE significantly lowers the barrier to bundle querying (§2).

**EXAMPLE 2 (PARAMETERIZING A PACKAGE QUERY).** *Continuing from Example 1, Morpheus recalls that strong new hires resemble recent hires at top universities, such as {Trinity, Cypher} (University X) and {Link, Niobe, Seraph} (University Y). These exemplars excel in Databases and AI with reasonable collective teaching experience. Morpheus computes their scores across 4 dimensions using the same embedding of Example 1 (Table 2). Guided by the aggregated scores, he parameterizes the underspecified query Q1 to obtain:*

```
Q2:  SELECT PACKAGE(*) FROM CANDIDATES
      SUCH THAT COUNT(*) = 2
      AND SUM(ai_score) BETWEEN 0.8 AND 1.2
      AND SUM(db_score) BETWEEN 1.0 AND 1.3
      AND SUM(teaching_score) BETWEEN 0.7 AND 1.3
      MAXIMIZE SUM(reco_score);
```

|                                      | AI         | Databases  | Teaching       | Recommendation |
|--------------------------------------|------------|------------|----------------|----------------|
|                                      | ai_score   | db_score   | teaching_score | reco_score     |
| <b>Example 1: University X hires</b> |            |            |                |                |
| Trinity                              | 0.6        | 0.5        | 0.3            | 0.7            |
| Cypher                               | 0.2        | 0.8        | 0.4            | 0.9            |
| <b>SUM</b>                           | <b>0.8</b> | <b>1.3</b> | <b>0.7</b>     | <b>1.6</b>     |
| <b>Example 2: University Y hires</b> |            |            |                |                |
| Link                                 | 0.9        | 0.3        | 0.4            | 0.6            |
| Niobe                                | 0.2        | 0.3        | 0.6            | 0.8            |
| Seraph                               | 0.1        | 0.4        | 0.3            | 0.7            |
| <b>SUM</b>                           | <b>1.2</b> | <b>1.0</b> | <b>1.3</b>     | <b>2.1</b>     |

Morpheus’ (implicit) intent

[0.8, 1.2] [1.0, 1.3] [0.7, 1.3] Maximize

Table 2: Example bundles help discover query parameters.

| bundle                        | ai         | db         | teaching   | reco | valid? |
|-------------------------------|------------|------------|------------|------|--------|
| b <sub>1</sub> {Smith, Jones} | 1.2        | 1.1        | 0.3 (Fail) | 1.3  | no     |
| b <sub>2</sub> {Smith, Neo}   | 1.2        | 0.9 (Fail) | 0.5 (Fail) | 1.2  | no     |
| b <sub>3</sub> {Smith, Brown} | 1.3 (Fail) | 0.8 (Fail) | 0.5 (Fail) | 1.3  | no     |
| b <sub>4</sub> {Jones, Neo}   | 0.8        | 1.2        | 0.6 (Fail) | 1.7  | almost |
| b <sub>5</sub> {Jones, Brown} | 0.9        | 1.1        | 0.6 (Fail) | 1.8  | almost |
| b <sub>6</sub> {Neo, Brown}   | 0.9        | 0.9 (Fail) | 0.8        | 1.7  | almost |

Table 3: No candidate bundle fully satisfies the constraints of Q2.

Example 2 highlights a scenario where a user cannot directly specify precise criteria via PaQL query parameters for bundle retrieval but can provide valid *examples* to implicitly convey the criteria. These examples can be used to automatically learn user preferences and translate them to query parameters. However, another key challenge remains, which we highlight in Example 3.

**EXAMPLE 3 (ENSURING QUERY FEASIBILITY TO OBTAIN RESULTS.).** *Morpheus issues Q2 to a package query execution engine [11], but it returns no result. A close inspection reveals that indeed none of the 2-candidate bundles fully satisfies Q2 (Table 3). However, the last three bundles “almost” satisfy the constraints. If the db\_score constraint had a slightly relaxed lower bound (0.9 instead of 1.0), then b<sub>6</sub> would be a valid bundle. Similarly, slightly relaxing the lower bound of the teaching\_score constraint (0.6 instead of 0.7) would result in two additional valid bundles: b<sub>4</sub> and b<sub>5</sub>. Then following the goal of maximizing the reco\_score, the bundle b<sub>5</sub> would be the final result.*

**Challenge 2: ensuring feasibility.** While BQbE addresses the first challenge, it brings forth a second one that is particular to bundle retrieval and absent from traditional single-tuple QbE: constraints inferred from example bundles may yield queries that are *infeasible* over the target data, returning empty answers [52, 53, 57], because aggregate constraints can be jointly unsatisfiable even when individually plausible, especially under distribution shift between the example and target domains. In Example 3, even with a fully formed PaQL query, with the desired constraints and specific parameters, it turned out to be *infeasible* over the target data (Morpheus’ University). This happened because the data distributions differed between the example domains (Universities X and Y) and the target domain (Morpheus’ University), a common scenario in practice [38, 63]. Existing package query execution systems [11] provide no further insight into how the query parameters interact with the target data, leaving users uncertain about how to tune the parameters to make the query feasible such that valid results are retrieved.

**Our solution to Challenge 2: data-aware relaxation of query parameters.** To ensure feasibility even when the original query is infeasible, we introduce *data-aware constraint relaxation techniques* that adapt the synthesized PaQL query to the target data by adjusting constraint bounds. By analyzing the data distribution of the target database, we identify which constraints act as bottlenecks that prevent “almost-valid” bundles from being retrieved. This analysis guides us to relax the appropriate constraints, while minimizing deviation from the user’s original intent.

**Why existing approaches fall short.** Three classes of relevant approaches exist, but all of them fail to support BQbE.

*PaQL and QbE systems.* PaQL engines [11] require fully parameterized queries and return empty results when infeasible; QbE systems [23, 88] reduce the parameter burden for single-tuple retrieval but do not extend to bundle queries with aggregate constraints.

*Top- $k$  retrieval over vector DBs.* For BQbE, an alternative solution is to treat the entire example bundle as a single query vector and retrieve top- $k$  similar tuples based on vector similarity [51, 66] to assemble the result bundle. However, this approach can miss globally optimal bundles and violate intended constraints. The main issue is that top- $k$  retrieval evaluates items independently using an implicit scoring function—without guarantees of global optimality—whereas bundle retrieval requires reasoning over combinations of tuples under globally enforced [87], multi-dimensional aggregate constraints. RAG systems inherit the same limitation, as they perform top- $k$  nearest-neighbor search over individual tuples (but not their combinations) before passing results to a generative model. Another alternative is to match each tuple in the example bundle independently and combine their top matches to form the result bundle; yet this approach breaks down when the example and target bundle sizes differ and still offers no optimality guarantees. In summary, bundle retrieval is fundamentally an NP-hard combinatorial optimization problem, and similarity-based greedy top- $k$  search is ill-suited to address it.

*Conversational LLMs.* An LLM prompted with examples (few-shot learning) could produce the bundle directly, but this requires reasoning over the entire target data—infeasible for large data—and solving a combinatorial constrained problem, which LLMs handle unreliably [76]. An LLM could instead produce code that extracts the bundle. Text-to-SQL is well-studied [25], but SQL operates on individual tuples and cannot natively express aggregate-constrained set selection. PaQL closes this gap; an LLM could plausibly synthesize PaQL queries similarly, though text-to-PaQL is less established. Even so, the bounds would reflect an opaque LLM-internal policy mapping the user’s examples to constraint values (e.g., element-wise min/max or mean $\pm\sigma$ ), with no verification. The LLM also has no native mechanism to detect infeasibility or relax bounds in an intent-preserving way; outputs are non-deterministic across versions, and per-query latency limits interactive use. In a pure NL interface, without examples to anchor the bounds, the user must rely on conversational refinement, which is imprecise and tedious [84].

**Desiderata.** Examples 1–3 and the limitations of alternative approaches motivate the desiderata of an ideal BQbE system:

**D1** Ensure a user-friendly interface for specifying intents, where users can provide examples to implicitly convey their bundle

query intents, without requiring any knowledge of the database schema and complex query syntax and parameters.

- D2** Accurately model the user intent implicit in the examples into a transparent intermediate representation with explicit parameters, such as a PaQL query.
- D3** Ensure feasibility of the synthesized PaQL query over the target database, yielding a non-empty result even when no bundle perfectly matches the user’s intent.
- D4** Provide an intuitive interface for iterative intent refinement, allowing users to adjust query constraints in a way that guarantees a valid result.

**Ex2BUNDLE.** We introduce Ex2BUNDLE, an example-driven data bundle retrieval system that (i) enables users to specify bundle query intent through example bundles, addressing D1; (ii) synthesizes a package query from these examples to transparently capture user intent, addressing D2; (iii) applies data-aware constraint relaxation to ensure feasibility of the synthesized query w.r.t. the target data domain—particularly when its distribution differs from that of the example data domain—addressing D3; and (iv) provides an intuitive interface for interactive intent refinement, addressing D4.

In the context of text summarization, BQbE arises as *focused text snippet extraction*: given a “focus” in the form of several pairs of source document and corresponding example summary—expressed as a set of extracted sentences from the document—the goal is to select a set of sentences from a target document that collectively forms an extractive summary consistent with those examples. Our preliminary work SuDocu (summarizing documents by example), published as a demonstration paper at VLDB [21], involved a simple solution for focused text snippet extraction. In this work, we significantly extend it to build a generalized and more robust framework to support bundle retrieval tasks across multiple domains.

**Contributions.** We make the following contributions:

- We introduce a novel paradigm for specifying *bundle retrieval intents through examples*, and highlight its broad applicability across real-world applications in several domains (§2).
- We formalize the problem of *example-driven bundle retrieval* and show how it translates to the package query framework (§3).
- We present Ex2BUNDLE, an end-to-end framework for *example-driven bundle retrieval* that synthesizes PaQL query constraints from user examples. We contribute a *data-aware constraint-bound relaxation technique* that ensures query feasibility. We further introduce design principles for an interactive slider-based interface for intent refinement, along with effective techniques to translate between sliders and constraint bounds (§4).
- We evaluate Ex2BUNDLE on two use cases—package queries over the TPC-H dataset [75] and focused text snippet extraction over real-world datasets [32, 82]—showing that Ex2BUNDLE achieves 100% constraint satisfaction while maintaining competitive objective scores, and outperforms retrieval and extractive baselines. Notably, Ex2BUNDLE remains competitive with LLM-based approaches without incurring any additional token or inference cost, while scaling to realistic workloads (§5).
- Our user study shows that Ex2BUNDLE achieves high user satisfaction, due to improved ease of use and customizable sliders, for the task of focused text snippet extraction (§6).

## 2 EXAMPLE USE CASES

We now present three real-world applications where example-driven intent specification lowers the barrier for data bundle retrieval: *supplier selection* for business, *focused snippet extraction* from text documents, and *playlist recommendation* in streaming services.

*Supplier selection for business expansion into a new country.* Bundle retrieval by example is valuable in business settings where an owner seeks to expand into a new country and must identify a set of new *suppliers* to establish relationships with. The objective is to select a bundle of about 100 suppliers from a large candidate pool over 10,000 suppliers (similar to the TPC-H [75] dataset). Manually constructing an optimal supplier set here is tedious and error-prone, as the selected suppliers must collectively satisfy a range of constraints. Formulating these requirements as an explicit package query is equally challenging: business owners often lack precise knowledge of the exact parameter values for their desired constraints in an unfamiliar market (e.g., acceptable price ranges, inventory availability, or financial stability thresholds). However, they can readily provide example bundles of suppliers drawn from countries where their business already operates successfully. Ex2BUNDLE can leverage these examples to infer the underlying implicit constraints, adjust them to the new supplier database of the target country, and retrieve an optimal set of suppliers that collectively satisfies the constraints.

*Focused text snippet extraction.* Another use case of Ex2BUNDLE is focused snippet extraction from text documents, also termed as personalized extractive summarization [82]. Here, the goal is to select (extract) a subset of sentences (snippet) from a text document that, collectively, best aligns with user’s information need (focus). For example, when a user wishes to extract sentences from a technical paper that are most relevant to their own research interests. While such intent can, in principle, be expressed in natural language, prior work [21, 82] shows that articulating subjective intents this way is difficult and often leads to user fatigue [84] due to iterative, back-and-forth intent refinement via conversations. Instead, users can provide a few example (document, snippet) pairs to implicitly communicate their focus [34, 67, 71, 83]. For instance, a journalist summarizing reports of all U.S. states with a particular focus—e.g., “basic” economic information, “moderate” coverage of education, and “strong” emphasis on technology—can highlight sentences from a few state’s reports to form example snippets. From these exemplars, Ex2BUNDLE can infer the journalist’s focus, transparently encode it as PaQL constraints, and apply them to new state reports to extract snippets with the same focus. Notably, for reports with substantially different structure or content, Ex2BUNDLE can minimally adjust these inferred constraints to ensure feasible snippet extraction.

*Playlist recommendation for streaming services.* In streaming services, users seek recommendations for *bundles* of items that collectively satisfy their preferences. Spotify, YouTube Music, and Netflix exemplify this through playlist/watchlist recommendations (e.g., Discover Weekly [68], Your Daily Discover [7], and Top Picks for You [3]), where the goal is to generate a set of songs/movies aligned with user interest. Existing approaches typically rely on item-level similarity—recommending songs similar to those in a user’s listening history—or on similarity to a single reference playlist. For instance, a user who frequently listens to AC/DC may receive playlists

dominated by rock songs, or playlists resembling those curated for users with similar histories. While effective in homogeneous preference settings, such approaches struggle to capture diverse tastes and is notoriously known to cause user frustration, as evident from complaints about lack of variety in Spotify’s Discover Weekly playlist [1]. Consider a user whose taste spans multiple genres—predominantly energetic rock, with a mix of heavy metal, some soft country songs, alongside occasional calm classical music. A playlist based only on rock similarity or a single reference list cannot capture such diversity. Moreover, explicitly specifying such nuanced preferences (e.g., how to balance genre, artist, mood, and tempo) is difficult, even with natural language or LLM-based interfaces. Example-driven bundle recommendation addresses this: users provide a few example playlists that reflect their desired diversity across genres, artists, tempo, and mood, and Ex2BUNDLE infers implicit constraints to generate playlists that collectively satisfy the inferred constraints.

## 3 PROBLEM FORMULATION

In this section, we formalize the problem of example-driven bundle retrieval. We begin by introducing how we model the *source* and *target data*—from which bundles are retrieved—and *example bundles*—which communicate the user’s intent. We then show how bundle retrieval can be naturally modeled within the package query framework and define the problem of *example-driven package query synthesis* for bundle retrieval, our focus in this paper. Table 4 provides a summary of notations used throughout the paper.

**Source and target data.** In example-driven bundle retrieval, each example bundle is taken from a corresponding source data. In Example 2, {Trinity, Cypher} is an example bundle from the source data of University X and {Link, Niobe, Seraph} from the source data of University Y. One key requirement here is that the source data for all the example bundles must have the same schema, to allow for effective intent discovery across the shared attributes.

We define a schema  $\mathbf{f}$  of a single table<sup>2</sup> over  $K$  numerical<sup>3</sup> attributes/features  $\{f_1, \dots, f_K\}$ . Each source data  $T_i^s$  must conform to the schema  $\mathbf{f}$ , denoted as  $T_i^s \models \mathbf{f}$ . In Table 2, the source data

<sup>2</sup>While our formalization focuses on a single-table schema, it is not an inherent limitation: we support relational databases by joining tables into a denormalized table, as shown in our experiments over the TPC-H dataset (Section 5).

<sup>3</sup>For unstructured data such as text, we derive numerical features through domain-specific methods such as topic modeling or learned vector embeddings (Section 4) to produce a structured representation under a fixed numerical schema.

| Symbol                                                     | Description                                                 |
|------------------------------------------------------------|-------------------------------------------------------------|
| $\mathbf{f} = \{f_1, \dots, f_K\}$                         | Common schema over $K$ attributes/features                  |
| $T_i^s \models \mathbf{f}$                                 | A source data over the schema $\mathbf{f}$                  |
| $t \models \mathbf{f}$                                     | A single tuple over the schema $\mathbf{f}$                 |
| $\mathbf{f}(t) = [f_1(t), \dots, f_K(t)] \in \mathbb{R}^K$ | Feature vector of tuple $t$                                 |
| $E_i \subseteq T_i^s$                                      | Example bundle for the source data $T_i^s$                  |
| $\mathbf{T}^s = \{T_1^s, \dots, T_N^s\}$                   | Set of $N$ source data                                      |
| $\mathbf{E} = \{E_1, \dots, E_N\}$                         | Set of $N$ user-provided example bundles                    |
| $T^q \models \mathbf{f}$                                   | A target data over $\mathbf{f}$ to retrieve a bundle from   |
| $B \models \mathbf{f}$                                     | A bundle over the schema $\mathbf{f}$                       |
| $\Theta = \{\{lb_j, ub_j\}\}_{j=1}^K$                      | Constraint bounds for $K$ attributes                        |
| $\mathbf{F}(B) = [F_1(B), \dots, F_K(B)]$                  | Feature profile of bundle $B$                               |
| $\mathbf{F}(B) \vdash \Theta$                              | $\mathbf{F}(B)$ satisfies the constraints given by $\Theta$ |
| $\sigma(t) \mapsto \mathbb{R}$                             | Domain-specific tuple-level scoring function                |

**Table 4: Table of notations. Bold letters denote sets or vectors.**



for all the Universities conform to the schema  $\{\text{ai\_score}, \text{db\_score}, \text{teaching\_score}, \text{reco\_score}\}$ . A tuple  $t$  over  $\mathbf{f}$ , denoted as  $t \models \mathbf{f}$ , is characterized by a feature vector  $\mathbf{f}(t) = [f_1(t), \dots, f_K(t)] \in \mathbb{R}^K$ . For instance, in Table 2, Trinity’s feature vector is  $[0.6, 0.5, 0.3, 0.7]$ . We use  $T^q \models \mathbf{f}$  to denote the *target data*, the data from which the user wants to extract their desired bundle from. In Example 2, Table 1 represents the target data.

**Example bundles.** In example-driven bundle retrieval, users convey their intent via a set of *example bundles*  $\mathbf{E} = \{E_1, \dots, E_N\}$ , where each  $E_i \subseteq T_i^s$  consists of a subset of tuples from the corresponding source data  $T_i^s$ . In Example 2,  $\mathbf{E} = \{\{\text{Trinity}, \text{Cypher}\}, \{\text{Link}, \text{Niobe}, \text{Seraph}\}\}$  and the source data set  $T^s = \{T_{\text{UoFX}}^s, T_{\text{UoFY}}^s\}$ .

### 3.1 Example-driven bundle retrieval

We are now ready to (informally) define our problem:

**PROBLEM 3.1 (EXAMPLE-DRIVEN BUNDLE RETRIEVAL).** *Given a set of source data  $T^s$  over the same schema  $\mathbf{f}$ , corresponding example bundles  $\mathbf{E}$ , where  $E_i \in \mathbf{E}$  is an example bundle for the source data  $T_i^s \in T^s$ , and a single target data  $T^q \models \mathbf{f}$ , identify a bundle  $B^* \subseteq T^q$  that (i) satisfies the “user intent” implicit in  $\mathbf{E}$  w.r.t  $T^s$  and (ii) is the “best” among such bundles.*

However, this problem is underspecified: it is unclear how to model the user intent and define “best” or the notion of optimality.

**3.1.1 Modeling user intent.** In its most general form, capturing user intent from example bundles may require arbitrarily complex models—non-linear constraints, distribution matching, or opaque learned representations. However, the use cases in Section 2 share a common structure: (i) the desired bundle is a *set* of items, (ii) quality of the bundle depends on some *aggregate* properties (e.g., total score) of the set, and (iii) the user’s intent is expressible as *bounds* on those aggregates. These are precisely the defining characteristics of a *package query* [11]—which retrieves subsets of tuples from relational tables that collectively satisfy (linear) aggregate constraints while optimizing a (linear) global objective—as illustrated by Q2 in Example 2. We therefore model user intent as *linear constraints* on the bundle’s *feature profile*, with an optional cardinality constraint on the bundle size. The benefit of this modeling is twofold: it provides *interpretability*—the resulting constraints are human-readable—and *tractability*—it maps to Integer Linear Programming (ILP) with well-established solvers and foundations [11].

**Feature profile of a bundle.** To capture the feature-wise properties of a bundle  $B$ , we define its *feature profile*  $\mathbf{F}$  as a vector obtained by aggregating, for each feature, the values of that feature across all tuples in  $B$ . Formally,

$$\mathbf{F}(B) = [F_1(B), \dots, F_K(B)], \text{ where } F_j(B) = \mathcal{A}_{t \in B} F_j(t)$$

Here,  $\mathcal{A}$  is an aggregate such as SUM. In Table 2, using SUM as the aggregate, the feature profile of the bundle  $\{\text{Trinity}, \text{Cypher}\}$  is  $[0.8, 1.3, 0.7, 1.6]$ —the column-wise sums for University X hires.

**Formulating constraints.** Following prior work in QbE [23], given a set of user-provided example bundles, we posit that the user’s intended result bundle would exhibit a feature profile similar to that of the example bundles. Accordingly, we model the user’s intent as a *set of bounded constraints over the feature profile* of the desired bundle, where the derivation of the bounds should be guided by

the feature profiles of the example bundles in  $\mathbf{E}$ , the source data set  $T^s$ , and the target data  $T^q$ . Formally:

$$\{1b_1 \leq F_1(B) \leq ub_1, \dots, 1b_K \leq F_K(B) \leq ub_K\}$$

where,  $1b_j$  and  $ub_j$  depend on  $\mathbf{E}$ ,  $T^s$ , and  $T^q$  for  $1 \leq j \leq K$

We use  $\Theta = \{\langle 1b_1, ub_1 \rangle, \dots, \langle 1b_K, ub_K \rangle\}$  to denote constraint bounds for all  $K$  attributes over the schema  $\mathbf{f}$ . The notation  $\mathbf{F}(B) \vdash \Theta$  denotes that the feature profile of the bundle  $B$  satisfies the constraint bounds specified by  $\Theta$ , i.e.,  $\forall 1 \leq j \leq K, 1b_j \leq F_j(B) \leq ub_j$ .

**3.1.2 Defining bundle optimality.** The second issue in Problem 3.1 is that, when multiple candidate bundles satisfy  $\Theta$ , a principled way is required to quantify their quality and ensure optimality. To this end, we assume knowledge of a tuple-level function  $\sigma(t) \mapsto \mathbb{R}$ , which is application-specific and is typically provided by a domain expert who configures the system for end users. We then define the quality of a bundle  $B$  by aggregating  $\sigma$  over all  $t \in B$ , which naturally fits the linear objective function optimized by the package query framework. For example, this could correspond to maximizing the total recommendation score in Example 1, or minimizing the word count in text summarization. Alternatively,  $\sigma$  can be learned from feedback signals reflecting users’ perceived quality of returned bundles in a human-in-the-loop setting, which is especially practical for playlist recommendation in music streaming (Section 2).

### 3.2 Example-driven package query synthesis

With our models of user intent and bundle optimality, which naturally align with the package query framework, we reformulate Problem 3.1 of example-driven bundle retrieval as that of synthesizing a package query—specifically, its parameters—from example bundles. The synthesized package query serves as a *mechanism* for efficiently retrieving the optimal result bundle.

Given a target data  $T^q$ , constraint bounds  $\Theta = \{\langle 1b_j, ub_j \rangle\}_{j=1}^K$ , and (optional) cardinality constraint bounds  $C = \langle 1b_c, ub_c \rangle$  we fix the following parameterized package query:

```
PQ( $T^q, \Theta, C$ ): SELECT PACKAGE(*) AS B FROM  $T^q$ 
SUCH THAT COUNT(B) BETWEEN  $1b_c$  AND  $ub_c$ 
AND  $F_j(B)$  BETWEEN  $1b_j$  AND  $ub_j \forall 1 \leq j \leq K$ 
MAXIMIZE  $\mathcal{A}_{t \in B} \sigma(t)$ 
```

Here,  $\mathcal{A}$  aggregates the tuples  $t \in B$  using  $\sigma$  to compute the quality of  $B$ , which the package query aims to maximize as its objective. Without loss of generality, a minimization objective can be expressed as a maximization objective by simply negating the objective. The choice of aggregation functions (including those used to compute the feature profile  $\mathbf{F}(B)$ ) and the scoring function  $\sigma$  is typically guided by the application domain. These are system parameters determined during setup and are not optimized over. We discuss how to choose the aggregation and scoring functions in Section 4.2. Also, without loss of generality, the cardinality constraint  $1b_c \leq \text{COUNT}(B) \leq ub_c$  can be subsumed into the feature-profile constraints by augmenting the feature profile with an additional dimension representing bundle cardinality. Hence, we do not explicitly include the cardinality constraint in the remainder of the paper.

In Example 2, Q2 is a package query over the schema  $\mathbf{f} = \{\text{ai\_score}, \text{db\_score}, \text{teaching\_score}\}$ , so  $K = 3$ . The aggregation used to compute the feature profile is SUM, and the objective is to maximize SUM(reco\_score) (i.e.,  $\sigma = \text{reco\_score}$ ).

While Morepheus manually derived the constraint bounds in  $\Theta$ , the resulting query was infeasible over the target data  $T^q$  representing his university. Thus, the main requirement is to determine the parameter  $\Theta$ , and use it to synthesize a package query that will retrieve the optimal bundle  $B \subseteq T^q$ , as specified in Problem 3.1.

**PROBLEM 3.2 (EXAMPLE-DRIVEN PACKAGE QUERY SYNTHESIS).** *Given a schema  $\mathbf{f}$ , a set of source data  $T^s = \{T_1^s, \dots, T_N^s\}$  where each  $T_i^s \models \mathbf{f}$ , corresponding example bundles  $\mathbf{E}$ , a target data  $T^q \models \mathbf{f}$ , and a tuple-level scoring function  $\sigma : t \mapsto \mathbb{R}$ , together with the corresponding aggregator  $\mathcal{A}$ , determine a synthesis function  $G : (\mathbf{E}, T^s, T^q) \mapsto \langle \mathbb{R}, \mathbb{R} \rangle^K$  to derive the constraint bounds  $\Theta$  such that:*

- (1) **Feasibility:** the package query  $PQ(T^q, \Theta)$  returns a non-empty result over  $T^q$  ensuring  $\exists B \subseteq T^q$  s.t.  $\mathbf{F}(B) \vdash \Theta$ .
- (2) **Optimality:** the retrieved bundle  $B^* = PQ(T^q, \Theta)(T^q)$  maximizes the objective, i.e.,  $B^* = \arg \max_{B \subseteq T^q \text{ s.t. } \mathbf{F}(B) \vdash \Theta} \mathcal{A} \sigma(t)$ .

The core challenge here is to determine the synthesis function  $G : (\mathbf{E}, T^s, T^q) \mapsto \langle \mathbb{R}, \mathbb{R} \rangle^K$  to determine the constraint bounds  $\Theta$  by capturing the intent implicit in  $\mathbf{E}$  while generalizing to the target data  $T^q$ . The challenge is that the distribution of  $T^q$  may differ from  $T^s$ : overly tight constraints (Example 2) can yield infeasible queries on the target data (overfitting), while overly relaxed constraints return bundles that deviate from user intent.

Section 4 describes the techniques to ensure query feasibility. Package queries avoid exhaustive search over  $T^q$  for efficiency, so the returned bundle may be suboptimal; however, our empirical results (Section 5) show the suboptimality is minimal.

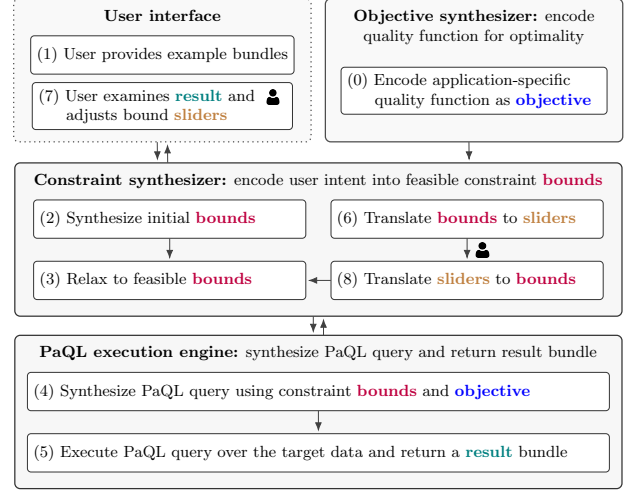
## 4 THE EX2BUNDLE FRAMEWORK

Ex2BUNDLE consists of three main components as shown in Figure 5: (§4.1) *constraint synthesizer*, which encodes user intent into *feasible* constraint **bounds**; (§4.2) *objective synthesizer*, which encodes an application-specific quality function as the **objective** of the PaQL query; and (§4.3) *PaQL execution engine*, which synthesizes and executes package queries to retrieve **result** bundles.

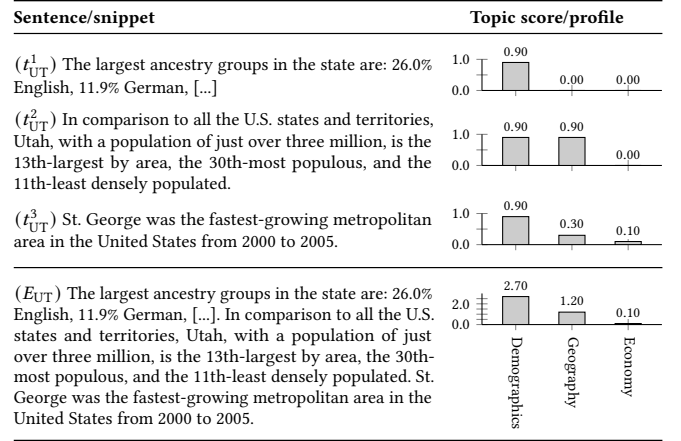
*Running example for focused text snippet extraction.* Throughout the rest of the paper, we use a running example centered on the task of *Focused Text Snippet Extraction* (FTSE), a primary application through which we evaluate Ex2BUNDLE (Section 5). Because Ex2BUNDLE requires numeric features and text is non-numeric, we apply *contextualized topic modeling* (CTM) [5] with sentence embeddings from SBERT [72] to map sentences into a numeric vector space of topic-based features. Under the resulting topic model defined over schema  $\mathbf{f}$ ,  $f_j(t) \in [0, 1]$  denotes the relevance of a sentence  $t$  to topic  $f_j$ , and  $\mathbf{f}(t)$  denotes the vector representation of  $t$  in the topic space  $\mathbf{f}$ . For ease of understanding, we use the following, more specific and domain-relevant terms for FTSE, instead of generic terms:

| General term       | FTSE term              | General term    | FTSE term     |
|--------------------|------------------------|-----------------|---------------|
| source/target data | source/target document | tuple           | sentence      |
| bundle             | snippet or summary     | schema          | topic model   |
| attribute/feature  | topic                  | feature profile | topic profile |

**EXAMPLE 4.** *Consider a topic model  $\mathbf{f}$  over Demographics, Geography, and Economy applied to documents describing U.S. states. Let  $T_{UT}^s$  denote the Utah document and  $E_{UT} = \{t_{UT}^1, t_{UT}^2, t_{UT}^3\} \subseteq T_{UT}^s$  be three*



**Figure 5: Ex2BUNDLE workflow:** before end-user interaction, (0) a domain expert defines a quality function, which Ex2BUNDLE encodes as the objective. During use, (1) the user provides example bundles, from which (2) Ex2BUNDLE synthesizes initial constraint bounds and (3) relaxes them to ensure feasibility. A PaQL query is (4) formed using these bounds and the objective and (5) executed to retrieve a result bundle. For further intent refinement, (6) an interactive slider interface is generated, (7) the user inspects results and adjusts sliders, (8) Ex2BUNDLE maps adjustments back to bounds, and repeats (3)–(5).



**Table 6: Topic scores for sentences  $t_{UT}^1$ ,  $t_{UT}^2$ , and  $t_{UT}^3$ ; and topic profile for the example snippet  $E_{UT} = \{t_{UT}^1, t_{UT}^2, t_{UT}^3\}$ .**

*sentences (Table 6). Under  $\mathbf{f}$ ,  $t_{UT}^1$  maps to  $\mathbf{f}(t_{UT}^1) = [0.90, 0.00, 0.00]$ , indicating a demographics focus;  $t_{UT}^2$  to  $[0.90, 0.90, 0.00]$ , covering demographics and geography; and  $t_{UT}^3$  to  $[0.90, 0.30, 0.10]$ , primarily demographics with weaker geography and economy associations.*

### 4.1 Encoding intent into feasible constraints

The constraint synthesizer (i) synthesizes initial, potentially infeasible, bounds from user examples to form constraints, (ii) relaxes the initial bounds to ensure feasibility, (iii) exposes the feasible bounds via a slider interface for iterative refinement, and (iv) translates the adjusted sliders back to feasible bounds after refinement.

---

**Algorithm 1:** Ex2BUNDLE bound relaxation algorithm

---

**Input** : Initial bounds  $\Theta_{\text{init}}$ , schema  $f$ , target data  $T^q$   
Relaxation parameters  $\mu$  and  $\tau$   
**Output** : Feasible bounds  $\Theta$  to construct the PaQL constraints

```

1  $\Theta \leftarrow \Theta_{\text{init}}$ 
2 #attempts  $\leftarrow 0$  // Initialize #attempts
3 while  $PQ(\Theta, T^q) = \emptyset$  // While query is infeasible
4 do
5    $\Theta_{\text{violated}} = \text{IdentifyViolations}(\Theta, T^q)$  // Violated constraints
6    $\Theta_{\text{relaxed}} = \emptyset$  // Relaxed constraints
7    $\rho \leftarrow \exp\left(\mu \cdot \max\left(1, \left\lfloor \frac{\text{\#attempts}++}{\tau} \right\rfloor\right)\right)$  // Relaxation step multiplier
8   foreach  $\langle lb_j, ub_j \rangle \in \Theta_{\text{violated}}$  do
9      $\epsilon \leftarrow \frac{F_j(T^q)}{|T^q|}$  // Relaxation step size
10     $lb_j \leftarrow \max(lb_j - \rho \cdot \epsilon, 0)$  // Decrease lower bound
11     $ub_j \leftarrow \min(ub_j + \rho \cdot \epsilon, F_j(T^q))$  // Increase upper bound
12     $\Theta_{\text{relaxed}} = \Theta_{\text{relaxed}} \cup \{\langle lb_j, ub_j \rangle\}$ 
13   $\Theta = (\Theta \setminus \Theta_{\text{violated}}) \cup \Theta_{\text{relaxed}}$  // Update constraint bounds
14 return  $\Theta$ 

```

---

**4.1.1 Synthesizing the initial bounds.** We extend SQuID [23], which learns user intent from example tuples, to a bundle-level granularity. Given example bundles  $E = \{E_i\}_{i=1}^N$ , we use SUM as the aggregate<sup>4</sup> to compute their feature profiles  $\{F(E_i)\}_{i=1}^N$ . We derive the initial constraint bounds  $\Theta_{\text{init}} = \{\langle lb_j, ub_j \rangle\}_{j=1}^K$  by taking the topic-wise minimum and maximum of the feature profiles of bundles in  $E$ :

$$lb_j = \min_{1 \leq i \leq N} F_j(E_i), \quad ub_j = \max_{1 \leq i \leq N} F_j(E_i)$$

**EXAMPLE 5.** Consider 3 example snippets  $E_{UT}$ ,  $E_{AZ}$ , and  $E_{WA}$  over the source documents  $T_{UT}^s$ ,  $T_{AZ}^s$ , and  $T_{WA}^s$  that represent Wikipedia pages for Utah, Arizona, and Washington, respectively. The topic profile of  $E_{UT}$ ,  $F(E_{UT}) = f(t_{UT}^1) + f(t_{UT}^2) + f(t_{UT}^3) = [2.70, 1.20, 0.10]$ , as shown in the last row of Table 6. The topic profiles for the example snippets, along with the initial constraint bounds are given below, which results in  $\Theta_{\text{init}} = \{\langle 1.50, 2.70 \rangle, \langle 0.90, 1.20 \rangle, \langle 0.10, 0.40 \rangle\}$ .

|             | $F_1$ (DEMOGRAPHICS) | $F_2$ (GEOGRAPHY) | $F_3$ (ECONOMY)   |
|-------------|----------------------|-------------------|-------------------|
| $F(E_{UT})$ | 2.70 $\uparrow$      | 1.20 $\uparrow$   | 0.10 $\downarrow$ |
| $F(E_{AZ})$ | 1.80                 | 0.90 $\downarrow$ | 0.40 $\uparrow$   |
| $F(E_{WA})$ | 1.50 $\downarrow$    | 1.00              | 0.30              |
| lb (min)    | 1.50                 | 0.90              | 0.10              |
| ub (max)    | 2.70                 | 1.20              | 0.40              |

**4.1.2 Bound relaxation to ensure feasibility.** Overly tight bounds can lead to infeasible queries that return no valid bundle over the target data (Example 3), particularly when the constraint bounds are misaligned with the target data’s feature distribution due to distributional shift. Infeasibility generally arises from two sources. The first is *insufficient feature coverage*, where the target data cannot satisfy a feature’s lower-bound constraint even if all tuples are selected. For instance, if the source data (e.g., Utah, Arizona, & Washington) and example snippets heavily emphasize the topic national parks (e.g., Bryce, Grand Canyon, Olympics), but the target document is Kansas, which contains no national parks, then its maximum possible contribution to that topic is 0. Any positive lower bound (e.g., 0.10) becomes unattainable, leading to infeasibility via lower-bound violation. The second source of infeasibility is *atomicity*,

which arises from the indivisibility of tuples. If selecting any single tuple already violates an upper bound, no feasible solution exists. For example, if every sentence in the target document has a topic score of at least 0.15 while the constraint’s upper bound is 0.10, feasibility is impossible since tuples cannot be fractionally selected.

**Bound relaxation algorithm.** We address infeasibility via *bound relaxation*, which iteratively widens constraint bounds until the resulting package query becomes feasible. Algorithm 1 outlines this process, which repeats until feasibility is achieved (lines 5–13). If the initial constraint bounds  $\Theta_{\text{init}}$  are infeasible (line 3), we first identify the violated constraints  $\Theta_{\text{violated}}$  (line 5) using ILP solver signals (e.g., IBM CPLEX [37]) used by the package query engine (§4.3). To preserve the original intent, we selectively relax only the violated constraints, while keeping all others unchanged.

Since it is unknown whether the upper or lower bound (or both) caused the violation, we relax both bounds symmetrically. We decrease  $lb_j$  toward 0 (line 10) and increase  $ub_j$  toward  $F_j(T^q)$  (line 11). We determine the relaxation amount at each iteration using  $\rho \cdot \epsilon$  (lines 10 & 11), where the step multiplier  $\rho$  (line 7) controls the relaxation rate based on the number of prior attempts, and the step size  $\epsilon$  (line 9) is a target-data-specific parameter.<sup>5</sup>

**Relaxation step multiplier,  $\rho$ .** It controls the relaxation rate and is defined as  $e^{\mu \cdot \max(1, \lfloor \text{\#attempts}/\tau \rfloor)}$ . Here,  $\mu$  (default 0.5) sets the base relaxation rate, while  $\tau$  (default 10) controls how frequently  $\rho$  is boosted. Consequently,  $\rho$  increases exponentially every  $\tau$  attempts, enabling faster escape from infeasible regions.

**Relaxation step size,  $\epsilon$ .** While  $\rho$  determines the rate of relaxation, different features exhibit varying score distributions within  $T^q$ . To account for this, we derive a feature-specific step size from the target data distribution and set  $\epsilon = \frac{F_j(T^q)}{|T^q|}$ , which corresponds to the average contribution of feature  $f_j$  per tuple in  $T^q$ .

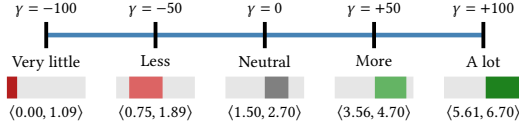
*Remark:* Although our relaxation algorithm is iterative (rather than a binary search or other principled optimization method), it achieves rapid convergence in practice. We empirically found that relaxation terminates in approximately 40 iterations and takes about 4 seconds on average, even when examples contain around 100 tuples. We provide more details in §5.5 and §5.6.

**EXAMPLE 6.** Consider the California document  $T_{CA}^q$  over 5 sentences ( $|T_{CA}^q| = 5$ ), where  $F_2(T_{CA}^q) = 0.60$ . For Geography ( $F_2$ ), the initially synthesized constraint is  $0.90 \leq F_2(B) \leq 1.20$  from Example 5. This is infeasible since the maximum achievable score, even when all sentences from  $T_{CA}^q$  are used to form the bundle, is 0.60, requiring relaxation. For the first 10 iterations,  $\rho = e^{0.5 \times 1} \approx 1.65$  and  $\epsilon = \frac{0.60}{5} = 0.12$ . Thus, the first iteration relaxes the bounds to  $\max(0.90 - 1.65 \times 0.12, 0) = 0.70 \leq F_2(B) \leq \min(1.20 + 1.65 \times 0.12, 0.60) = 0.60$ . This is still infeasible, so the second iteration yields  $\max(0.70 - 1.65 \times 0.12, 0) = 0.50 \leq F_2(B) \leq \min(0.60 + 1.65 \times 0.12, 0.60) = 0.60$ , at which point the constraint becomes feasible and relaxation stops. Note that, the upper bound is not relaxed due to it already being at max.

**4.1.3 Refining intent via bound sliders.** A key usability requirement (desideratum D4 in §1) for Ex2BUNDLE is an intuitive user interface for iterative intent refinement. A straightforward approach is to let

<sup>4</sup>SUM is preferred since AVG cannot distinguish single- from multi-tuple bundles, and MAX/MIN are outlier-sensitive. Our framework can support any linear aggregate.

<sup>5</sup>The step size  $\epsilon$  is analogous to *gradient* and the step multiplier  $\rho$  to *learning rate* in stochastic gradient descent. However, constraint violation is not differentiable, since it yields zero gradients in the interior and undefined gradients at the boundary.



**Figure 7: Slider for DEMOGRAPHICS.** Users can adjust topic emphasis from  $-100$  (very little) to  $+100$  (a lot). Each slider position maps to specific constraint bounds; the neutral position ( $0$ ) corresponds to the feasible bounds synthesized (and relaxed) from user examples.

users directly edit the upper and lower bounds of the synthesized constraints. While this offers full transparency and suits experts, it reduces usability for non-expert users, who do not know how to map their intent to the corresponding constraint bounds.

Motivated by studies showing that humans are better at relative judgments (more vs. less) than absolute ones (assigning ratings) [74], we introduce a single-parameter slider [15, 24, 40, 64, 69, 79] that abstracts raw bounds (Figure 7). Each slider position corresponds to a pair of upper and lower bounds. For example, for GEOGRAPHY, the neutral point  $0$  corresponds to the feasible bounds  $\langle 1.50, 2.70 \rangle$  synthesized from user examples (Example 5). The slider further encodes other bounds into a scale from  $-100$  to  $+100$  around the neutral point to allow fine-grained adjustments. Upon reviewing the result bundle, users simply indicate whether they want more (less) relevance to the topic by moving the slider right (left). Beyond enabling relative adjustments, this design reduces the number of controllable parameters, consistent with human-computer interaction findings that simpler control spaces are cognitively less demanding [36].

**Mapping between slider positions and bounds.** We use  $\gamma_j$  to denote the position of the slider for topic  $f_j$ . We map the initial feasible bounds  $\langle lb_j, ub_j \rangle$ —synthesized from user examples—to the midpoint of the slider at  $\gamma_j=0$ . For a target document  $T^q$ , the minimum and maximum attainable bundle scores for topic  $f_j$  are  $0$  and  $F_j(T^q)$ , respectively. Let us use  $mx$  to denote  $F_j(T^q)$  for simplicity. These two extreme values— $0$  and  $mx$ —map to the lower bound of the left-most slider position and upper bound of the right-most slider position. However, a key challenge here is that both sides of the midpoint of the slider have equal number of positions (100 on each side), however, the distance between the initial feasible bounds and the two extreme bounds are not necessarily equal. Thus, for slider positions left of the midpoint, we determine the *lower* bound by interpolating based on the position’s relative distance between the left-most endpoint ( $\gamma_j=-100$ ) and the midpoint ( $\gamma_j=0$ ). Symmetrically, for positions right of the midpoint, we determine the *upper* bound by interpolating based on the position’s relative distance between the right-most endpoint ( $\gamma_j=+100$ ) and the midpoint ( $\gamma_j=0$ ). Assuming a constant width  $w$  for all bounds, the mappings are as follows:

| Slider Position       | Lower bound                                | Upper bound                            | $\gamma_j$ |
|-----------------------|--------------------------------------------|----------------------------------------|------------|
| (left-most)           | $0$                                        | $w$                                    | $-100$     |
| (left of the center)  | $lb_j \cdot \frac{(100-x)}{100}$           | $lb_j \cdot \frac{(100-x)}{100} + w$   | $-x$       |
| (center)              | $lb_j$                                     | $ub_j$                                 | $0$        |
| (right of the center) | $ub_j + \frac{(mx-ub_j) \cdot x}{100} - w$ | $ub_j + \frac{(mx-ub_j) \cdot x}{100}$ | $+x$       |
| (right-most)          | $mx - w$                                   | $mx$                                   | $+100$     |

A question still remains: how to determine the width of the bounds  $w$  in the above mappings? Should it be constant for all slider positions? We use the width of the neutral position ( $ub_j - lb_j$ ) as a guide to derive the bound width of slider position at  $\gamma_j=x$  with the formula:  $w(x) = e^{-\frac{\alpha \cdot |x|}{100}} \cdot (ub_j - lb_j)$ .

Here,  $\alpha$  is a tunable parameter between  $0$  and  $\infty$ , which determines how aggressively bound widths are narrowed as the slider moves farther away from the neutral position. When  $\alpha=0$ , all slider positions use the same width as the neutral position, i.e.,  $(ub_j - lb_j)$ . As  $\alpha$  increases, the width shrinks more rapidly as the slider moves away from the neutral position, resulting in increasingly tighter bounds for extreme values of  $\gamma_j$ . Regardless, Ex2BUNDLE automatically “snaps” the slider to a feasible position in case the user’s adjustment leaves it at an infeasible state. Note that the bounds represented by different slider positions can be overlapping (Figure 7).

**EXAMPLE 7.** For feature  $f_1$  (DEMOGRAPHICS), the synthesized bounds from the user examples are  $\langle 1.50, 2.70 \rangle$  (Example 5). The maximum score along this feature is  $F_1(T_{CA}^q) = 6.70$ . Figure 7 illustrates the corresponding slider: the neutral position ( $\gamma=0$ ) is mapped to  $\langle 1.50, 2.70 \rangle$ . At  $\gamma=+100$ , the upper bound is fixed to the maximum value  $6.70$ . With  $\alpha=0.1$ , the width is  $w(+100) = e^{-\frac{0.1 \cdot |100|}{100}} \cdot (2.70 - 1.50) = e^{-0.1} \cdot 1.20 \approx 1.09$ . The lower bound is therefore  $6.70 - 1.09 = 5.61$ , yielding the bounds  $\langle 5.61, 6.70 \rangle$  for  $\gamma=+100$ . At  $\gamma=-50$ , the lower bound is interpolated as  $1.50 \cdot \frac{(100-50)}{100} = 0.75$ . The width is  $w(-50) = e^{-\frac{0.1 \cdot |50|}{100}} \cdot 1.20 = e^{-0.05} \cdot 1.20 \approx 1.14$ . Adding this width to the lower bound yields the upper bound  $0.75 + 1.14 = 1.89$ , so  $\gamma=-50$  maps to  $\langle 0.75, 1.89 \rangle$ .

**Bound relaxation after slider interaction.** Since slider adjustments may re-introduce infeasibility, we re-relax the bounds afterwards using Algorithm 1. Once feasible bounds are obtained, we snap the slider to the position whose bounds are closest to the feasible bounds and subsume them, ensuring transparency to the user. Unlike the initial relaxation, we have an advantage here since we only need to relax bounds for a single constraint, with all other constraint bounds fixed. Therefore, if previously feasible bounds  $\langle lb_j^*, ub_j^* \rangle$  are known, we use it as an additional reference during relaxation. Specifically, we reduce  $lb_j$  until  $\max(0, lb_j^*)$  (line 10 of Algorithm 1) and increase  $ub_j$  until  $\min(ub_j^*, F_j(T^q))$  (line 11). Additionally, if the user requests termination of relaxation before the process finishes, we move the slider to the closest feasible bounds.

Overall, our slider interface offers the following benefits: (1) *simplicity* by reducing the number of controls, (2) *semantic clarity* through an intuitive scale from less (negative) to more (positive), (3) *feasibility preservation* by automatically snapping user adjustments to a feasible state, and (4) *reversibility* by allowing users to reset to the original (or any intermediate) intent. Users can still adjust the bounds directly if they wish, but the slider provides a more intuitive alternative that bypasses the complex underlying numeric bounds. We evaluate the effectiveness of our design choices for the slider interface in our user study (Section 6).

## 4.2 Devising a quality function for optimality

As discussed in Section 3.1.2, the quality function defines the notion of optimality used to break ties among multiple valid bundles that satisfy the constraints. This becomes particularly important as bounds relaxation often yields multiple valid bundles. The quality function is application-specific and is typically specified by a domain expert prior to user interaction with an instance of Ex2BUNDLE, as shown in step (0) in Figure 5. Given (i) a tuple-level scoring function  $\sigma : t \mapsto \mathbb{R}$ , which assigns a numeric score to a tuple  $t$ , and (ii) an application-specific aggregate function  $\mathcal{A}$ ,



the quality of a candidate bundle  $B$ ,  $quality(B) = \mathcal{A}_{t \in B} \sigma(t)$ . Below, we present example quality functions—along with suitable aggregators—for the three applications discussed in Section 2.

*Supplier selection.* *Cost* and *reliability* are common quality functions when selecting a bundle of suppliers for business expansion into a new country. For cost, a natural aggregate is SUM, with the objective of minimizing total cost, i.e.,  $\sum_{t \in B} cost(t)$ . For reliability, suitable aggregates include AVG or MIN, depending on the preference: maximizing average reliability, or maximizing the minimum reliability among the selected suppliers.

*Focused text snippet extraction.* *Informativeness* and *conciseness* are common quality functions when extracting snippets or summaries from a text document. Sentence-level informativeness can be modeled using term frequency within a sentence  $t$ , weighted by inverse document frequency in the corpus (TF-IDF), which assigns higher scores to sentences containing rarer words with respect to the corpus. Alternatively, informativeness can be approximated using structural signals such as the presence of numbers, citations, or hyperlinks, which often indicate factual or reference-rich content. Conciseness can be modeled using the word count or character count of a sentence, with the objective of minimizing the total number of words or characters in a snippet. In both cases, the most appropriate aggregate is SUM. Alternatively, conciseness can be modeled by minimizing the number of sentences in a snippet, using COUNT as a bundle-level quality function that returns the snippet cardinality.

*Playlist recommendation.* *Popularity* and *diversity* are common quality functions in playlist recommendation. Popularity can be modeled using the number of weekly plays or chart rankings of a song, with the objective of maximizing overall popularity. A suitable aggregate in this case is AVG, i.e.,  $AVG_{t \in B} popularity(t)$ . Diversity can be modeled using pairwise dissimilarity between songs based on features such as genre, mood, or artist characteristics. This can be captured via a distance function over song features, with the objective of maximizing diversity within the playlist. A suitable aggregate is MIN over all pairwise distances, i.e.,  $\min_{t_i, t_j \in B, i \neq j} distance(t_i, t_j)$ , and maximizing this ensures that even the most similar pair of songs in the playlist remains sufficiently dissimilar.<sup>6</sup>

### 4.3 Package query execution

For the package query execution engine, we use PaQL [11], a query engine for declarative package queries. PaQL translates queries into Integer Linear Programs (ILPs), which are then solved using off-the-shelf solvers such as IBM CPLEX [37]. For large target data with many tuples, solving the ILP becomes computationally expensive due to the combinatorial nature of the search space. To address this, PaQL includes the SKETCHREFINE algorithm, a scalable method with a guaranteed  $(1 + \epsilon)^6$  approximation. SKETCHREFINE first clusters similar tuples to form a compact sketch of the data, solves the ILP over the cluster representatives, and then maps the solution back to the original tuples to construct the final bundle.

Ex2BUNDLE formulates a PaQL query that encodes the constraint bounds  $\Theta$  (§4.1) as constraints over packages (analogous to snippets in FTSE or bundles in general) and defines the optimization

objective using the scoring function  $\sigma$  together with the aggregate  $\mathcal{A}$  (§4.2), as shown in Section 3.2. It then executes the package query, and, if feasible, returns the optimal bundle. The PaQL engine also informs the constraint synthesizer (§4.1) about any violated constraints when the query is infeasible in line 5 of Algorithm 1.

EXAMPLE 8. For the constraint bounds  $\Theta = \{\langle 1.50, 2.70 \rangle, \langle 0.50, 0.60 \rangle, \langle 0.10, 0.40 \rangle\}$  (derived in Example 5 and relaxed in Example 6) and the target document  $T_{CA}^q$ , we want the snippet with the minimum total word count. The function  $word\_count : t \mapsto \mathbb{N}^+$  returns the number of words in a sentence  $t$ . The synthesized PaQL query is as follows:

```
PQ( $T_{CA}^q, \Theta$ ): SELECT PACKAGE(*) AS B FROM  $T_{CA}^q$ 
  SUCH THAT  $F_1(B)$  BETWEEN 1.50 AND 2.70
  AND  $F_2(B)$  BETWEEN 0.50 AND 0.60
  AND  $F_3(B)$  BETWEEN 0.10 AND 0.40
  MINIMIZE SUM $_{t \in B}$  word_count( $t$ )
```

## 5 EXPERIMENTAL RESULTS

In this section, we present experimental results addressing the following research questions. (RQ1) Does Ex2BUNDLE satisfy user-specified bundle constraints, and how does this compare to constraint-agnostic baselines? (§5.2) (RQ2) Do Ex2BUNDLE’s retrieved bundles align with user intent, and how does this compare to extractive and LLM-based baselines? (§5.3 & §5.4) (RQ3) How often is constraint relaxation triggered, and how does it scale with the number of sentences in the example snippets? (§5.5) (RQ4) How does Ex2BUNDLE’s runtime scale with the number of sentences in the example snippets? (§5.6)

### 5.1 Experimental setup

**Implementation.** Ex2BUNDLE uses a Python backend—with Gensim for LDA (frequency-based topic modeling for FTSE), SBERT [72] for CTM (semantics-aware topic modeling), IBM CPLEX 20.1 via docplex for ILP solving, and Flask 2.0 for serving—and a JavaScript/Bootstrap frontend with an interactive slider interface.<sup>7</sup> We ran experiments on a 2019 MacBook Pro with a 2.4 GHz Quad-Core Intel Core i5 CPU and 16 GB of 2133 MHz LPDDR3 RAM.

**Datasets.** We evaluate Ex2BUNDLE in two applications: supplier selection over the TPC-H dataset, and focused text snippet extraction over SUBSUME and CNN/DailyMail. For TPC-H [75], we synthesize a view from Supplier (100 rows), Partsupp (8,000 rows), and Nation (25 rows) at scale factor 0.01, with 5 attributes including price, availability, balance, region\_europe, and region\_america. SUBSUME [82] contains 275 (user, intent) pairs, each with 8 manually curated summaries; we use 5 as user input and hold out the remaining 3 as a test set. CNN/DailyMail [32] contains news articles with human-written highlights, adapted using ChatGPT-4o to map each highlight to its closest sentences in the source article. We use a subset of 100 articles drawn from the ACL2020 split, with an average of 39 sentences per article (median 35, range 7–151). Within each category, we use 5 articles as user examples and 3 as targets, matching the train/test protocol used for SUBSUME.

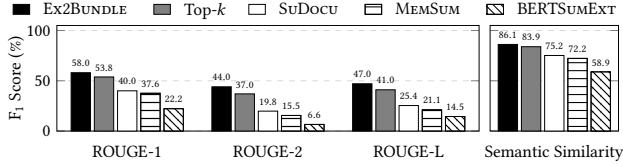
**Baselines.** Section 5.2 introduces task-specific baselines for RQ1. For RQ2–RQ4: *Top-k* selects the top- $k$  sentences from the target document by SBERT [72] cosine similarity to the example sentences ( $k$  is the average example size). *SuDocu* [21] is our prior system for

<sup>6</sup>While in this paper we assume a linear, tuple-wise scoring function, Ex2BUNDLE can support quadratic objective via CPLEX’s quadratic programming or MILP linearization.

<sup>7</sup>Ex2BUNDLE source code: <https://github.com/kuangfei-long/ex2bundle>

| System           | CSR (%) $\uparrow$ | Average Objective Score $\uparrow$ | Runtime (s) $\downarrow$ |
|------------------|--------------------|------------------------------------|--------------------------|
| Random           | 70.0               | 10.48                              | <b>0.0005</b>            |
| Greedy           | 33.3               | <b>15.61</b>                       | <u>0.0024</u>            |
| <b>Ex2BUNDLE</b> | <b>100.0</b>       | <u>11.72</u>                       | 0.0207                   |

**Table 8: Ex2BUNDLE achieves full constraint satisfaction with a competitive objective score. Bold = best; underlined = second-best.**



**Figure 9: For focused text snippet extraction on the SUBSUME dataset, Ex2BUNDLE outperforms all other baselines across all metrics.**

focused text snippet extraction, using LDA [6] topic distributions. *BERTSUMEXT* [50] is a widely-adopted extractive summarizer; we adapt it for example-driven extraction by pre-filtering the document to sentences semantically similar to the examples. *MEMSUM* [27] is a state-of-the-art reinforcement-learning extractive summarizer for long documents; we adapt it via the same pre-filtering as *BERTSUMEXT*. *ChatGPT-4o* [59] is OpenAI’s LLM; we provide the target document and example summaries via file attachment and prompt it to extract snippets.

**Metrics.** We use the following metrics:

- **Constraint Satisfaction Rate (CSR):** percentage of inferred constraints satisfied by the retrieved bundle, averaged across runs.
- **Objective score:** composite of TPC-H attributes maximizing the sum of price, availability, balance.
- **ROUGE-1/2/L** [46]:  $F_1$  scores measuring unigram, bigram, and longest-common-subsequence overlap between retrieved and ground-truth snippets.
- **Semantic Similarity (SS):** cosine similarity between average SBERT embeddings of retrieved and ground-truth snippets.

More details on the setup, including additional description of the datasets and baselines, are in our technical report [14].

## 5.2 Constraint satisfaction performance

We evaluate constraint satisfaction in the BQbE setting for the task of supplier selection (RQ1) on the TPC-H dataset. We construct 3 example supplier bundles representing diverse procurement strategies (conservative, price-focused, balanced); Ex2BUNDLE infers constraints from these examples per §4.1, synthesizes a PaQL query, and executes it to retrieve a result bundle. We compare Ex2BUNDLE against *Greedy* (top- $k$  tuples by objective score, ignoring constraints) and *Random* (uniform  $k$  tuples), where  $k$  is the average example bundle size, averaged over 5 runs.

As shown in Table 8, Ex2BUNDLE satisfies all constraints (CSR = 100%) with a competitive average objective score (11.72). Greedy achieves the highest objective score (15.61) but satisfies on average only 33.3% of the inferred constraints, indicating that objective-only optimization fails to enforce explicit constraint requirements. Random selection has a higher CSR (70.0%) than Greedy with a lower objective score (10.48), highlighting the importance of jointly optimizing both constraint satisfaction and objective.

| Intent                                                                    | System     | Semantic Similarity $\uparrow$ |
|---------------------------------------------------------------------------|------------|--------------------------------|
| Intent 1: What about this state’s arts and culture attracts you the most? | ChatGPT-4o | 0.64                           |
|                                                                           | Ex2BUNDLE  | <b>0.88</b>                    |
| Intent 2: What are some of the most interesting things about this state?  | ChatGPT-4o | 0.65                           |
|                                                                           | Ex2BUNDLE  | <b>0.76</b>                    |

**Table 10: Ex2BUNDLE outperforms ChatGPT-4o on both SUBSUME intents; the advantage narrows on the more general one.**

### Key Takeaways

- Ex2BUNDLE achieves full constraint satisfaction with a competitive objective score.
- Greedy fails on constraint satisfaction, Random on objective score; Ex2BUNDLE jointly optimizes both.

## 5.3 Retrieval-based FTSE performance

For RQ2, we evaluate Ex2BUNDLE on SUBSUME against Top- $k$ , SuDocu, BERTSUMEXT, and MEMSUM. We report ROUGE-1/2/L  $F_1$  scores and SBERT-based semantic similarity (SS) between retrieved and ground-truth snippets, together with per-query runtime.

Figure 9 shows that Ex2BUNDLE consistently outperforms all baselines on ROUGE and semantic similarity. SuDocu, which is example-driven, outperforms the pure extractive summarizers MEMSUM and BERTSUMEXT (which lack example-driven intent modeling), but its topic-modeling approach is inferior to Ex2BUNDLE and Top- $k$ , both of which use semantics-aware topic modeling.

### Key Takeaways

- Ex2BUNDLE outperforms all baselines on ROUGE and SS.
- Systems that use semantics-aware topic modeling (Ex2BUNDLE) and Top- $k$  outperform SuDocu and pure extractive summarization (BERTSUMEXT, MEMSUM).

## 5.4 LLM-based summarization performance

Ex2BUNDLE is LLM-free: its latency scales with corpus rather than model size, it applies to private data where LLM pipelines cannot run, and it incurs no per-query inference cost. The tradeoff is potential accuracy loss; for RQ2, we test this against ChatGPT-4o on two settings: *focused intents* from SUBSUME (specific needs) and *generic intents* from CNN/DailyMail (open-ended summaries).

**Focused intent.** We compare Ex2BUNDLE against ChatGPT-4o (RAG, few-shot) on two intents from SUBSUME—one more specific, one less so—to see how each tool handles different levels of intent specificity (Table 10). Ex2BUNDLE outperforms ChatGPT-4o on both (SS=0.88 vs 0.64 for the more specific Intent 1; 0.76 vs 0.65 for the less specific Intent 2). The advantage narrows on the less specific intent—Ex2BUNDLE drops from 0.88 to 0.76 while ChatGPT-4o stays roughly flat (0.64 to 0.65)—but Ex2BUNDLE leads in both.

**Generic intent.** On CNN/DailyMail, we compare against ChatGPT-4o (few-shot) across four categories (Politics, Crime, Sports, & Lifestyle). ChatGPT-4o leads on all ROUGE and SS metrics in every category (Table 11); on Politics, for instance, ChatGPT-4o achieves ROUGE-L 0.4814 and SS 0.7965, vs. 0.2335 and 0.6272 for Ex2BUNDLE (similar gaps elsewhere). This is expected: generic news snippets have broad intent that example-driven retrieval cannot capture [86], marking the regime where Ex2BUNDLE is not the appropriate tool.

| Category  | System     | ROUGE-1 $\uparrow$ | ROUGE-2 $\uparrow$ | ROUGE-L $\uparrow$ | Semantic Similarity $\uparrow$ |
|-----------|------------|--------------------|--------------------|--------------------|--------------------------------|
| Politics  | ChatGPT-4o | <b>0.5504</b>      | <b>0.4403</b>      | <b>0.4814</b>      | <b>0.7965</b>                  |
|           | Ex2BUNDLE  | 0.3235             | 0.1590             | 0.2335             | 0.6272                         |
| Crime     | ChatGPT-4o | <b>0.4844</b>      | <b>0.3513</b>      | <b>0.4019</b>      | <b>0.7601</b>                  |
|           | Ex2BUNDLE  | 0.3246             | 0.1499             | 0.2204             | 0.5829                         |
| Sports    | ChatGPT-4o | <b>0.5170</b>      | <b>0.4219</b>      | <b>0.4548</b>      | <b>0.8022</b>                  |
|           | Ex2BUNDLE  | 0.3148             | 0.1743             | 0.2330             | 0.6255                         |
| Lifestyle | ChatGPT-4o | <b>0.5060</b>      | <b>0.3812</b>      | <b>0.4294</b>      | <b>0.7949</b>                  |
|           | Ex2BUNDLE  | 0.3136             | 0.1304             | 0.1982             | 0.6016                         |

**Table 11: ChatGPT-4o outperforms Ex2BUNDLE in generic intent alignment across CNN/DailyMail news categories.**

#### Key Takeaways

- Ex2BUNDLE outperforms ChatGPT-4o on focused intents.
- The advantage narrows on less specific focused intents.
- On generic extraction, ChatGPT-4o leads, marking the regime where example-driven retrieval is less effective.

## 5.5 Relaxation analysis

We analyze relaxation behavior (RQ3) on 50 target documents varying example sizes from 5 to 45 sentences. For each query, Ex2BUNDLE synthesizes constraints from the example bundles (§4.1) and tries to retrieve a bundle from the target document satisfying those constraints; if no feasible solution exists, Ex2BUNDLE iteratively relaxes the violated constraints until feasibility is reached (§4.1.2). About 17% of queries are initially infeasible, independent of example size, indicating that infeasibility is driven by the match between target content and example-derived bounds rather than by the size of examples. For infeasible queries, Figure 12 shows the distribution of relaxation iterations needed to reach feasibility, which grows with example size: more example sentences yield tighter bounds, requiring more relaxation iterations to reach feasibility.

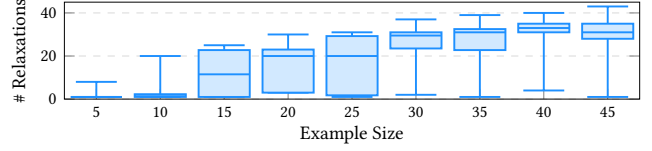
#### Key Takeaways

- About 17% of queries require constraint relaxation, independent of example size.
- More example sentences yield tighter constraint bounds, requiring more relaxation iterations to reach feasibility.

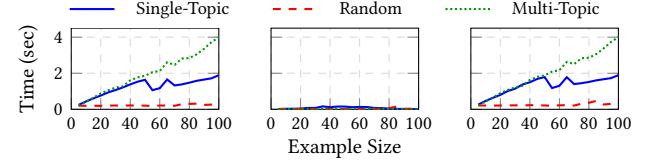
## 5.6 Scalability analysis

We evaluate Ex2BUNDLE’s scalability with respect to example-bundle complexity in terms of their size, i.e., the number of sentences  $k$  (RQ4). Since scalability in target document size is inherited from the PaQL framework [11], we focus on the example complexity. On CNN/DailyMail, we vary the example size  $k$  from 5 to 100 sentences in increments of 5, using 5 source documents and 45 target documents per setting. We synthesize examples via three sampling strategies: *single-topic* (top- $k$  sentences for one randomly selected topic), *multi-topic* (iteratively sampling topics at random until  $k$  sentences are collected), and *random* (uniform random  $k$  sentences from the document). We report *learning time* (average time to compute bounds and synthesize the PaQL query from examples) and *retrieval time* (average time to execute the PaQL query and retrieve the snippet from the target document).

Figure 13 shows that learning time (left) dominates total time (right), exceeding retrieval time (center) across all strategies, since



**Figure 12: Relaxation Analysis for Ex2BUNDLE across varying size of examples. The number of relaxations triggered increases with the number of example sentences.**



**Figure 13: Average learning (left), retrieval (center), and total times (right) while varying the example sizes in the example snippets for single-topic, random, and multi-topic sampling strategies.**

computing bounds is more expensive than PaQL execution. The completion time remains within 5 seconds even at 100-sentence examples, with total time scaling linearly. Among strategies, multi-topic has the highest learning time (it computes bounds across a more diverse snippet), while random has the lowest (treating selection like independent Top- $k$  rather than a combinatorial bundle).

#### Key Takeaways

- Learning time dominates total time across all sampling strategies; bound computation dominates the runtime.
- Ex2BUNDLE runs in under 5 seconds even for 100-sentence example snippets, with total time scaling linearly.

Experiments evaluating the impact of quality function variants on retrieval performance, prompt details, and extensive comparisons against baselines are in our technical report [14].

## 6 INTENT SLIDERS: A USER STUDY

To evaluate the effectiveness of the intent-refinement slider interface of Ex2BUNDLE for FTSE, we conducted a within-subjects study with 20 participants recruited via Amazon Mechanical Turk, fluent in English and experienced with document search. Participants were tasked with identifying their preferred US state for relocation using Wikipedia articles, comparing Ex2BUNDLE against SBERT (Top- $k$  baseline, §5.3), a widely-used embedding model in vector databases that outperforms other baselines in our evaluation. We exclude non-BQbE baselines (e.g., conversational LLMs), which underperform on FTSE (Table 10). Slider parameter  $\alpha$  was set to 0.1. Following a mixed-methods approach [26], participants were randomly assigned to start with either tool (anonymized) to mitigate transfer effects. Each participant was assigned to 25 randomly selected states per tool and was instructed to provide snippets for at least 2 states with no upper limit. While interacting with Ex2BUNDLE, they could select sentences, search using keywords, and refine intents by adjusting sliders. In SBERT, they could select sentences and search using keywords as well, but intent adjustment had to be done via manual addition/removal of sentences from example snippets.

We quantitatively analyzed the data by comparing the participants’ ratings of summary quality, ease of use, and daily usage

preference for both SBERT and Ex2BUNDLE. We also compared the number of interactions with the interface (e.g., selecting examples, adjusting sliders, requesting snippet retrieval, etc.) between the two tools to understand how participants engaged with them.

**Participants’ satisfaction.** On task completion, participants rated each tool on summary quality, ease of use, and daily usage preference. Ex2BUNDLE received higher ratings than SBERT on all three: (i) 12/20 preferred Ex2BUNDLE’s summaries (8/20 for SBERT), (ii) 16/20 found Ex2BUNDLE easier to use (only 4/20 for SBERT), and (iii) 11/20 preferred Ex2BUNDLE for daily use (9/20 for SBERT).

**Participants’ interactions.** To understand how participants interacted with the tools, we categorized interactions into six types: *Search*: keyword queries, *Selection*: example selection, *Update*: example modification, *Learn*: intent learning, *Retrieval*: backend retrieval, and *Slider*: interactive parameters. Averaging interaction counts per user, SBERT users performed about 3.1 times more *Search* interactions than Ex2BUNDLE users. In contrast, Ex2BUNDLE users had more *Slider* interactions and consequently about 2.1 times more *Retrieval* interactions, as Ex2BUNDLE automatically retrieves bundles upon slider adjustments. Ex2BUNDLE’s interactive sliders encouraged more iterative refinement of results, while SBERT’s lack of interactivity led to more manual searching.

Participants provided feedback on their experience with both tools. The majority used the interactive sliders to refine their intents: *“I raised the slider up so that the output summaries would prioritize information that pertained to the economy and climate of each state since those were the most important factors for me.”* However, some preferred the simplicity of SBERT, while others noted difficulty giving the system examples, highlighting a limitation of the example-based approach: *“I found the original text hard to focus on due to being one large text block [...] trying to find stuff like food and cuisine didn’t necessarily pick up on things without knowing exact keywords.”*<sup>8</sup>

## 7 RELATED WORK

**Programming by-example** [29, 30] is a paradigm where users provide examples to express their intent and has seen significant success in databases, particularly in data discovery-by-example [43, 55, 62] and query-by-example (QbE) [8, 17, 22, 23, 61, 65, 88], which allows users to express their query intent through examples rather than formal query languages. QPlain [17] incorporates explanations with data provenance, some works prune the search space for efficient query synthesis [8, 65] or generalize the output [61], and recently SQuID [22, 23] semantically synthesizes SQL queries from user-provided examples rather than relying on structural similarity. However, all existing QbE systems are designed to retrieve individual items and do not account for the combinatorial nature of bundle retrieval. Ex2BUNDLE addresses the bundle retrieval problem through the lens of QbE, where users express their intent through example bundles, as opposed to example tuples.

**Focused text snippet extraction**, also known as query-focused extractive summarization, allows users to specify keywords or queries to guide snippet selection [39, 45, 47], but requires users to explicitly formulate their intents. RAG-based approaches [2, 31, 44, 48] can also support snippet extraction via few-shot prompting [10],

but are not designed for extracting bundles that jointly satisfy multiple constraints. Recently, *faceted query-by-example* approaches leverage representative documents as an example query, additionally conditioned on explicit facets (constraints) to extract snippets across multiple constraints [18, 56, 77]. However, existing systems either require users to specify constraints as keywords (e.g., Method in scientific articles) [18] or as natural language queries [77], or rely on opaque vector matching that provides no interpretable constraint values [56]. Moreover, because these baselines score snippets independently, their objective functions fail to capture the joint constraint satisfaction required for bundle retrieval.

**Bundle retrieval** is a fundamental task in database that involves selecting an optimal set of items subject to a collection of constraints [11, 13, 19, 54, 78]. It has also been studied extensively in recommender systems, where the goal is to generate a bundle that collectively satisfies user preferences across various domains, such as e-commerce [49, 70, 80], entertainment [12, 60], and travel planning [20, 28, 85]. However, these systems rely on greedy heuristics or approximate generative models to generate bundles and struggle to enforce strict constraints. In relational databases, prior work on package queries addresses the efficiency of bundle query performance [11, 54], but assumes users can already express a well-formed query. Diversification-based retrieval approaches [13, 19, 78] also retrieve bundles that collectively satisfy user constraints. However, they typically operate over a limited set of constraints such as cardinality or diversity scores. In contrast, Ex2BUNDLE addresses the upstream challenge of automatically synthesizing multi-constraint package queries from user-provided examples.

## 8 SUMMARY AND FUTURE DIRECTIONS

We introduced Ex2BUNDLE, an example-driven framework for bundle retrieval that enables users to specify intents through example bundles. Ex2BUNDLE synthesizes package queries from examples and employs principled constraint relaxation to ensure feasibility. Our experiments and user study show that Ex2BUNDLE captures user intent and outperforms other baselines in intent alignment.

Several directions extend the framework. Manual example selection is burdensome, especially for complex intents or large data; an adaptive example recommendation that infers preference patterns from prior selections would lower this barrier. The current relaxation widens bounds symmetrically once violations are detected; more principled policies, such as minimal or preference-weighted relaxation, could better preserve user intent. Ex2BUNDLE currently optimizes linear objectives over per-tuple scores; native PaQL support for pairwise or quadratic objectives (e.g., bundle diversity for playlists) would broaden expressiveness. Specification of the quality function requires domain expertise; inferring both objective and constraints from example bundles—a problem related to inverse optimization [33]—would remove the domain-expertise requirement. Inference relies on example bundles alone; when examples are few or fail to convey aspects of intent that are better specified using natural-language modifiers such as “like these examples but with stronger emphasis on diversity”, a hybrid example-driven inference with LLM-based reasoning under symbolic soundness guarantees would extend coverage while preserving bound verifiability.

<sup>8</sup>More details on the user study are in our technical report [14].



## REFERENCES

- [1] 2020. Discover Weekly keeps giving me the same genre that I'm completely sick of. <https://community.spotify.com/t5/Your-Library/Discover-Weekly-keeps-giving-me-the-same-genre-that-I-m-tid-p/5065068>.
- [2] 2026. LangChain. <https://github.com/langchain-ai/langchain> Accessed: 2026-01-07.
- [3] 2026. Recommendations: Figuring out how to bring unique joy to each member. <https://research.netflix.com/research-area/recommendations>.
- [4] Jinze Bai, Chang Zhou, Junshuai Song, Xiaoru Qu, Weiting An, Zhao Li, and Jun Gao. 2019. Personalized Bundle List Recommendation. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*. ACM, 60–71.
- [5] Federico Bianchi, Silvia Terragni, Dirk Hovy, Debora Nozza, and Elisabetta Persini. 2021. Cross-lingual Contextualized Topic Models with Zero-shot Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Association for Computational Linguistics, Online, 1676–1683.
- [6] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet Allocation. *J. Mach. Learn. Res.* 3 (2003), 993–1022.
- [7] Jay Bongolito. 2025. Google is testing a new 'Daily Discover' feed on YouTube Music. <https://tech.yahoo.com/streaming/articles/google-testing-daily-discover-feed-170300058.html>.
- [8] Angela Bonifati, Radu Ciucanu, and Slawek Staworko. 2016. Learning Join Queries from User Examples. *ACM Trans. Database Syst.* 40, 4 (2016), 24:1–24:38.
- [9] Virginia Braun and Victoria Clarke. 2012. *Thematic analysis*. American Psychological Association.
- [10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.
- [11] Matteo Brucato, Juan Felipe Beltran, Azza Abouzied, and Alexandra Meliou. 2016. Scalable Package Queries in Relational Database Systems. *Proc. VLDB Endow.* 9, 7 (2016), 576–587.
- [12] Jianxin Chang, Chen Gao, Xiangnan He, Depeng Jin, and Yong Li. 2023. Bundle Recommendation and Generation With Graph Neural Networks. *IEEE Trans. Knowl. Data Eng.* 35, 3 (2023), 2326–2340.
- [13] Whanhee Cho and Anna Fariha. 2026. Data-Semantics-Aware Recommendation of Diverse Pivot Tables. *Proc. ACM Manag. Data* 4, 1, Article 23 (April 2026), 28 pages.
- [14] Whanhee Cho, Kuangfei Long, Mahmood Jasim, Matteo Brucato, Alexandra Meliou, Peter J. Haas, and Anna Fariha. 2026. Example-Driven Intent Synthesis for Constrained Data Bundle Retrieval: Focused Text Snippet Extraction and Beyond (Technical Report). *Preprint* (2026). [https://users.cs.utah.edu/~afariha/ex2bundle\\_tech\\_rep.pdf](https://users.cs.utah.edu/~afariha/ex2bundle_tech_rep.pdf)
- [15] John Joon Young Chung and Eytan Adar. 2023. PromptPaint: Steering Text-to-Image Generation Through Paint Medium-like Interactions. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (San Francisco, CA, USA) (UIST '23)*. Association for Computing Machinery, New York, NY, USA, Article 6, 17 pages.
- [16] Matthew JC Crump, John V McDonnell, and Todd M Gureckis. 2013. Evaluating Amazon's Mechanical Turk as a tool for experimental behavioral research. *PloS One* 8, 3 (2013), e57410.
- [17] Daniel Deutch and Amir Gilad. 2016. QPlain: Query by explanation. In *32nd IEEE International Conference on Data Engineering, ICDE 2016, Helsinki, Finland, May 16-20, 2016*. IEEE Computer Society, 1358–1361.
- [18] Heejin Do, Sangwon Ryu, Jonghwi Kim, and Gary Lee. 2025. Multi-Facet Blending for Faceted Query-by-Example Retrieval. In *ACL*. 28577–28590.
- [19] Marina Drosou and Evaggelia Pitoura. 2012. DisC diversity: result diversification based on dissimilarity and coverage. *Proc. VLDB Endow.* 6, 1 (2012), 13–24.
- [20] Shih Hsin Fang, Eric Hsueh-Chan Lu, and Vincent S. Tseng. 2014. Trip Recommendation with Multiple User Constraints by Integrating Point-of-Interests and Travel Packages. In *IEEE 15th International Conference on Mobile Data Management, MDM 2014, Brisbane, Australia, July 14-18, 2014 - Volume 1*. IEEE Computer Society, 33–42.
- [21] Anna Fariha, Matteo Brucato, Peter J. Haas, and Alexandra Meliou. 2020. SuDocu: Summarizing Documents by Example. *Proc. VLDB Endow.* 13, 12 (2020), 2861–2864.
- [22] Anna Fariha, Lucy Cousins, Narges Mahyar, and Alexandra Meliou. 2026. Example-driven semantic-similarity-aware query intent discovery: Empowering users to cross the SQL barrier through query by example. *Inf. Syst.* 138 (2026), 102687.
- [23] Anna Fariha and Alexandra Meliou. 2019. Example-Driven Query Intent Discovery: Abductive Reasoning using Semantic Similarity. *Proc. VLDB Endow.* 12, 11 (2019), 1262–1275.
- [24] Rohit Gandikota, Joanna Materzynska, Tingrui Zhou, Antonio Torralba, and David Bau. 2024. Concept Sliders: LoRA Adaptors for Precise Control in Diffusion Models. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XL (Lecture Notes in Computer Science)*. Springer, 172–188.
- [25] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (2024), 1132–1145.
- [26] Jennifer C Greene, Valerie J Caracelli, and Wendy F Graham. 1989. Toward a conceptual framework for mixed-method evaluation designs. *Educational evaluation and policy analysis* 11, 3 (1989), 255–274.
- [27] Nianlong Gu, Elliott Ash, and Richard H. R. Hahnloser. 2022. MemSum: Extractive Summarization of Long Documents Using Multi-Step Episodic Markov Decision Processes. (2022), 6507–6522. <https://doi.org/10.18653/v1/2022.ACL-LONG.450>
- [28] Qi Gu, Jian Cao, and Yancen Liu. 2022. CSBR: A Compositional Semantics-Based Service Bundle Recommendation Approach for Mashup Development. *IEEE Trans. Serv. Comput.* 15, 6 (2022), 3170–3183.
- [29] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2011, Austin, TX, USA, January 26-28, 2011*. ACM, 317–330.
- [30] Sumit Gulwani and Prateek Jain. 2017. Programming by Examples: PL Meets ML. In *Programming Languages and Systems - 15th Asian Symposium, APLAS 2017, Suzhou, China, November 27-29, 2017, Proceedings (Lecture Notes in Computer Science)*, Vol. 10695. Springer, 3–20.
- [31] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 3929–3938.
- [32] Karl Moritz Hermann, Tomás Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching Machines to Read and Comprehend. In *NeurIPS*. 1693–1701.
- [33] Clemens Heuberger. 2004. Inverse Combinatorial Optimization: A Survey on Problems, Methods, and Results. *Journal of Combinatorial Optimization* 8, 3 (2004), 329–361.
- [34] Po Hu, Donghong Ji, Chong Teng, and Yujing Guo. 2012. Context-Enhanced Personalized Social Summarization. In *Proceedings of COLING 2012. The COLING 2012 Organizing Committee, Mumbai, India, 1223–1238*.
- [35] Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. 2021. Efficient Attentions for Long Document Summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 1419–1436.
- [36] Andy Hunt, Marcelo M. Wanderley, and Matthew Paradis. 2002. The Importance of Parameter Mapping in Electronic Instrument Design. In *New Interfaces for Musical Expression, NIME-02, Proceedings, Dublin, Ireland, May 24-26, 2002*. Media Lab Europe, 149–154.
- [37] IBM ILOG CPLEX Optimization Studio [n.d.]. IBM ILOG CPLEX Optimization Studio. <https://www.ibm.com/docs/en/icos>.
- [38] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. 2015. Overview of Data Exploration Techniques. In *SIGMOD*. ACM, 277–281.
- [39] Hal Daumé III and Daniel Marcu. 2006. Bayesian Query-Focused Summarization. In *ACL 2006, 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, Sydney, Australia, 17-21 July 2006*, Nicoletta Calzolari, Claire Cardie, and Pierre Isabelle (Eds.). The Association for Computer Linguistics. <https://doi.org/10.3115/1220175.1220214>
- [40] Rahul Jain, Amit Goel, Koichiro Niinuma, and Aakar Gupta. 2025. AdaptiveSliders: User-aligned Semantic Slider-based Editing of Text-to-Image Model Output. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 541, 27 pages.
- [41] Chris Kedzie, Kathleen McKeown, and Hal Daumé III. 2018. Content Selection in Deep Learning Models of Summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 1818–1828.
- [42] Shahedul Huq Khandkar. 2009. Open coding. *University of Calgary* 23, 2009 (2009).
- [43] Akash Khatri, Mahathir Mohammad, and El Kindi Rezig. 2025. Sort it Like You Mean It: Discovering Semantically Interesting Attribute Augmentations to Sort Tables. *Proc. VLDB Endow.* 18, 12 (2025), 5427–5430. <https://doi.org/10.14778/3750601.3750688>
- [44] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim

- Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- [45] Yanran Li and Sujian Li. 2014. Query-focused Multi-Document Summarization: Combining a Topic Model with Graph-based Semi-supervised Learning. In *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*. Dublin City University and Association for Computational Linguistics, Dublin, Ireland, 1197–1207.
- [46] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [47] Marina Litvak and Natalia Vanetik. 2017. Query-based summarization using MDL principle. In *Proceedings of the multiling 2017 workshop on summarization and summary evaluation across source types and genres*. 22–31.
- [48] Jerry Liu. 2022. *LlamaIndex*. [https://github.com/jerryliu/llama\\_index](https://github.com/jerryliu/llama_index)
- [49] Xiaohao Liu, Jie Wu, Zhulin Tao, Yunshan Ma, Yinwei Wei, and Tat-Seng Chua. 2025. Fine-tuning Multimodal Large Language Models for Product Bundling. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1, KDD 2025, Toronto, ON, Canada, August 3-7, 2025*. ACM, 848–858.
- [50] Yang Liu and Mirella Lapata. 2019. Text Summarization with Pretrained Encoders. (2019), 3728–3738. <https://doi.org/10.18653/V1/D19-1387>
- [51] Xuan Lu, Sifan Liu, Bochao Yin, Yongqi Li, Xinghao Chen, Hui Su, Yaohui Jin, Wenjun Zeng, and Xiaoyu Shen. 2025. MultiConIR: Towards Multi-Condition Information Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 13471–13494.
- [52] Gang Luo. 2006. Efficient Detection of Empty-Result Queries. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*. ACM, 1015–1025.
- [53] Gang Luo. 2006. Efficient Detection of Empty-Result Queries. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*. ACM, 1015–1025.
- [54] Anh L. Mai, Pengyu Wang, Azza Abouzied, Matteo Brucato, Peter J. Haas, and Alexandra Meliou. 2024. Scaling Package Queries to a Billion Tuples via Hierarchical Partitioning and Customized Optimization. *Proc. VLDB Endow.* 17, 5 (2024), 1146–1158.
- [55] Mir Mahathir Mohammad and El Kindi Rezig. 2026. Qualitative Join Discovery in Data Lakes using Examples. *Proc. ACM Manag. Data* 4, 1, Article 68 (April 2026), 28 pages. <https://doi.org/10.1145/3786682>
- [56] Sheshera Mysore, Arman Cohan, and Tom Hope. 2022. Multi-Vector Models with Textual Guidance for Fine-Grained Scientific Document Similarity. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, Seattle, United States, 4453–4470.
- [57] Arnab Nandi and H. V. Jagadish. 2011. Guided Interaction: Rethinking the Query-Result Paradigm. *Proc. VLDB Endow.* 4, 12 (2011), 1466–1469.
- [58] Ani Nenkova, Sameer Maskey, and Yang Liu. 2011. Automatic Summarization. In *The 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Proceedings of the Conference, 19-24 June, 2011, Portland, Oregon, USA - Tutorial Abstracts*. The Association for Computer Linguistics, 3.
- [59] OpenAI. 2023. GPT-4 Technical Report. [arXiv:2303.08774 \[cs.CL\]](https://arxiv.org/abs/2303.08774)
- [60] Apurva Pathak, Kshitiz Gupta, and Julian J. McAuley. 2017. Generating and Personalizing Bundle Recommendations on Steam. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, Shinjuku, Tokyo, Japan, August 7-11, 2017*. ACM, 1073–1076.
- [61] Fotis Psallidas, Bolin Ding, Kaushik Chakrabarti, and Surajit Chaudhuri. 2015. S4: Top-k Spreadsheet-Style Search for Query Discovery. In *SIGMOD*. ACM, 2001–2016.
- [62] El Kindi Rezig, Anshul Bhandari, Anna Fariha, Benjamin Price, Allan Vanterpool, Vijay Gadepally, and Michael Stonebraker. 2021. DICE: Data Discovery by Example. *Proc. VLDB Endow.* 14, 12 (2021), 2819–2822.
- [63] Thibault Sellam and Martin L. Kersten. 2013. Meet Charles, big data query advisor. In *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org.
- [64] Vidya Setlur, Sarah E. Battersby, Melanie Tory, Rich Gossweiler, and Angel X. Chang. 2016. Eviza: A Natural Language Interface for Visual Analysis. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology, UIST 2016, Tokyo, Japan, October 16-19, 2016*. ACM, 365–377.
- [65] Yanyan Shen, Kaushik Chakrabarti, Surajit Chaudhuri, Bolin Ding, and Lev Novik. 2014. Discovering queries based on example tuples. In *SIGMOD*. ACM, 493–504.
- [66] Yunxiao Shi, Haoning Shang, Xing Zi, Wujiang Xu, Yue Feng, and Min Xu. 2025. Answering Narrative-Driven Recommendation Queries via a Retrieve-Rank Paradigm and the OCG-Agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 13181–13202.
- [67] Aviv Slobodkin, Niv Nachum, Shmuel Amar, Ori Shapira, and Ido Dagan. 2023. SummHelper: Collaborative Human-Computer Summarization. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023 - System Demonstrations, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 554–565.
- [68] Spotify Advertising Team. 2020. Five years of discovery and engagement through Discover Weekly. <https://ads.spotify.com/en-US/news-and-insights/five-years-of-discovery-and-engagement-through-discover-weekly/>. Accessed: 2026-03-17.
- [69] Hendrik Strobelt, Albert Webson, Victor Sanh, Benjamin Hoover, Johanna Beyer, Hanspeter Pfister, and Alexander M. Rush. 2023. Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models. *IEEE Trans. Vis. Comput. Graph.* 29, 1 (2023), 1146–1156.
- [70] Wenqi Sun, Ruobing Xie, Junjie Zhang, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Generative Next-Basket Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*. ACM, 737–743.
- [71] Hiroaki Takatsu, Takahiro Kashikawa, Koichi Kimura, Ryota Ando, and Yoichi Matsuyama. 2021. Personalized Extractive Summarization Using an Ising Machine Towards Real-time Generation of Efficient and Coherent Dialogue Scenarios. In *Proceedings of the 3rd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, Online, 16–29.
- [72] Nandan Thakur, Nils Reimers, Johannes Daxenberger, and Iryna Gurevych. 2021. Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise Sentence Scoring Tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 296–310.
- [73] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.
- [74] Louis L. Thurstone. 1974. *A Law of Comparative Judgment* (1st ed.). Routledge. 12 pages.
- [75] Transaction Processing Performance Council (TPC). [n.d.]. *TPC-H: Decision Support Benchmark*. Technical Report. TPC. <http://www.tpc.org/tpch/>
- [76] Joseph Tso, Preston Schmittou, Quan Huynh, and Jibrán Hutchins. 2026. ConstraintBench: Benchmarking LLM Constraint Reasoning on Direct Optimization. *arXiv preprint arXiv:2602.22465* (2026).
- [77] Jianyou Wang, Kaicheng Wang, Xiaoyue Wang, Prudhviraj Naidu, Leon Bergen, and Ramamohan Paturi. 2023. DORIS-MAE: scientific document retrieval using multi-level aspect-based queries. , Article 1668 (2023), 16 pages.
- [78] Yue Wang, Alexandra Meliou, and Jerome Miklau. 2018. RC-Index: Diversifying Answers to Range Queries. *Proc. VLDB Endow.* 11, 7 (2018), 773–786.
- [79] Zhijie Wang, Yuheng Huang, Da Song, Lei Ma, and Tianyi Zhang. 2024. PromptCharm: Text-to-Image Generation through Multi-modal Prompting and Refinement. In *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11-16, 2024*. ACM, 185:1–185:21.
- [80] Penghui Wei, Shaoguo Liu, Xuanhua Yang, Liang Wang, and Bo Zheng. 2022. Towards Personalized Bundle Creative Generation with Contrastive Non-Autoregressive Decoding. In *SIGIR '22: The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, July 11 - 15, 2022*. ACM, 2634–2638.
- [81] Wen Xiao and Giuseppe Carenini. 2019. Extractive Summarization of Long Documents by Combining Global and Local Context. In *EMNLP-IJCNLP*. Association for Computational Linguistics, 3009–3019.
- [82] Nishant Yadav, Matteo Brucato, Anna Fariha, Oscar Youngquist, Julian Killingback, Alexandra Meliou, and Peter Haas. 2021. SubSumE: A Dataset for Subjective Summary Extraction from Wikipedia Documents. In *New Frontiers in Summarization Workshop*. <https://summarization2021.github.io>
- [83] Rui Yan, Jian-Yun Nie, and Xiaoming Li. 2011. Summarize What You Are Interested In: An Optimization Framework for Interactive Personalized Summarization. In *EMNLP, ACL*, 1342–1351.
- [84] Hamed Zamani, Bhaskar Mitra, Everest Chen, Gord Lueck, Fernando Diaz, Paul N. Bennett, Nick Craswell, and Susan T. Dumais. 2020. Analyzing and Learning from User Interactions for Search Clarification. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*. ACM, 1181–1190.
- [85] Jiale Zhang, Mingqian Ma, Xiaofeng Gao, and Guihai Chen. 2024. Encoder-Decoder Based Route Generation Model for Flexible Travel Recommendation. *IEEE Trans. Serv. Comput.* 17, 3 (2024), 905–920.
- [86] Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen R. McKeown, and Tatsunori B. Hashimoto. 2024. Benchmarking Large Language Models for News Summarization. *Trans. Assoc. Comput. Linguistics* 12 (2024), 39–57.
- [87] Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. 2025. Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs. In *ICLR*.
- [88] Moshé M. Zloof. 1975. Query-by-Example: the Invocation and Definition of Tables and Forms. In *Proceedings of the International Conference on Very Large Data Bases, September 22-24, 1975, Framingham, Massachusetts, USA*. ACM, 1–24.

## APPENDIX A: USER STUDY

To evaluate Ex2BUNDLE, we conducted a within-subjects study with 20 participants recruited via Mechanical Turk. Participants were tasked with identifying their preferred state for relocation based on Wikipedia articles, using both a baseline (SBERT) and Ex2BUNDLE. We followed a mixed-methods approach [26] to analyze quantitative and qualitative data from the study. The study protocol was approved by the institutional review board (IRB) of University of Massachusetts Amherst.

### A.1 User Study Settings

**Participants.** We recruited 20 participants (P01-P20) through Amazon Mechanical Turk [16]. Our inclusion criteria stipulated that the participants must reside in North America, where this study was conducted, and be fluent in English. All participants were qualified Amazon Master Workers, demonstrating a high success level across a wide range of tasks. Participants were compensated with \$25.

**Procedure.** We followed a single-session within-subject study protocol (Figure A1). Each study session began with collecting participants' consent. Participants then watched a 5-minute unskippable video tutorial on using the tool before performing their tasks. After the tutorial, participants completed a pre-study questionnaire to gather their information-seeking purpose, practice, tool usage, time spent, strategy, and ratings of the quality of information. Then, participants were randomly assigned to start with either SBERT or Ex2BUNDLE to negate the transfer effect. During the study, participants interacted with an interface, Figure A2, to perform their tasks. The interface was designed to allow users to input examples, review and modify them, adjust the importance of different topics using sliders only for Ex2BUNDLE, and view the retrieved snippets. We anonymized the tools' names to obscure which tool each was using.

Participants were asked to perform a decision-making task: to identify which state in the United States (US) they would relocate to and live in if they were offered their dream job there. We provided them with Wikipedia articles on 25 randomly generated US states. Therefore, each participant worked with two different sets of 25 states across the two tools. We instructed them to read and input at least 2 state snippets with no upper limit. They were also free to use the assigned tool however they wanted.

After working with one tool, we asked the participants to perform the same task with the other tool, but this time, Wikipedia articles were provided on 25 other states. This was done to negate potential unwanted bias and transfer effect from one tool to the other. After completing tasks with both tools, we asked them a set of post-study questions to compare SBERT and Ex2BUNDLE regarding their summary quality, ease of use, and extracted snippets. We also asked them if given the choice, which tool they would use to perform their usual information-seeking tasks.

**Data Collection and Analysis.** We collected both quantitative and qualitative data. Quantitative data include usage logs, interactions, and time spent working with the tool components. We also collected responses to the Likert scale as well as qualitative answers to open-ended questions pre-study, post-task, and post-study questionnaires. We analyzed the quantitative data by comparing the participants' ratings such as the summary quality, ease of use, and daily usage of the tools. We analyzed the qualitative data using

thematic analysis [9]. Two members of the research team independently coded data from five randomly selected participants using the open coding method [42] using spreadsheets. The coders discussed, resolved disagreements, and consolidated their codes into a codebook containing the representative set of codes. Then the coders used this codebook to individually code the data from all 20 participants, adding new codes as they emerged. Finally, they compiled the codes into themes across five discussion sessions. The themes were discussed and finalized by the research team.<sup>9</sup>

### A.2 Pre-study Questionnaire

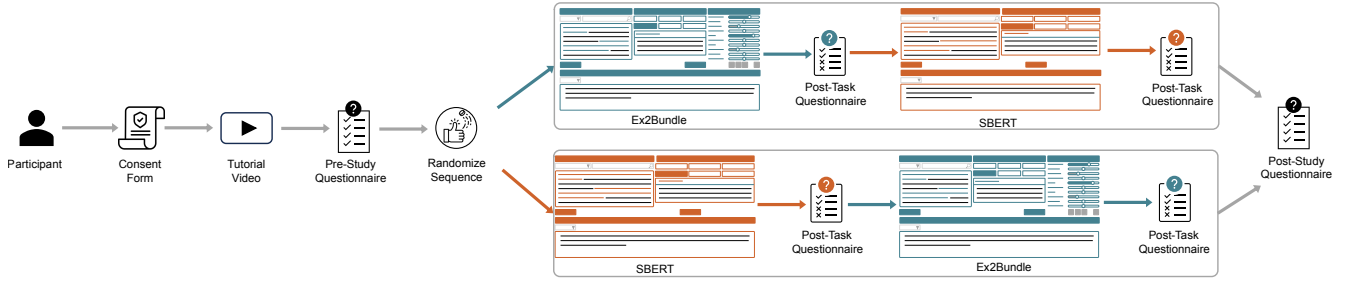
In the pre-study questionnaire, we asked participants about their information-seeking purpose, practice, tool usage, time spent, strategy, and ratings of the quality of information. We show bar charts of the results for four of the questions in Figure A3. The majority of participants (11/20) reported frequently searching across multiple documents or websites. In an information-searching session, the majority of participants (10/20) spent 15-30 minutes, while 2/20 spent <15 minutes, and 5/20 spent 30-45 minutes and 2/20 spent 45-60 minutes, or 1/20 60+ minutes. When asked about the unexpected results, 10/20 said rarely, 7/20 said neutral, and 3/20 said often. This suggests that most participants do not often encounter unexpected results during their information-seeking process. Finally, 8/20 rated the quality of the information gathered after their search process as good. Another 6/20 rated average, while 6/20 rated above average. In summary, the pre-study questionnaire results suggest that most participants frequently search across multiple documents or websites, spend a moderate amount of time on information-seeking sessions, and do not often encounter unexpected results. The quality of information gathered is generally rated as good or above average by most participants.

### A.3 Post-study Questionnaire

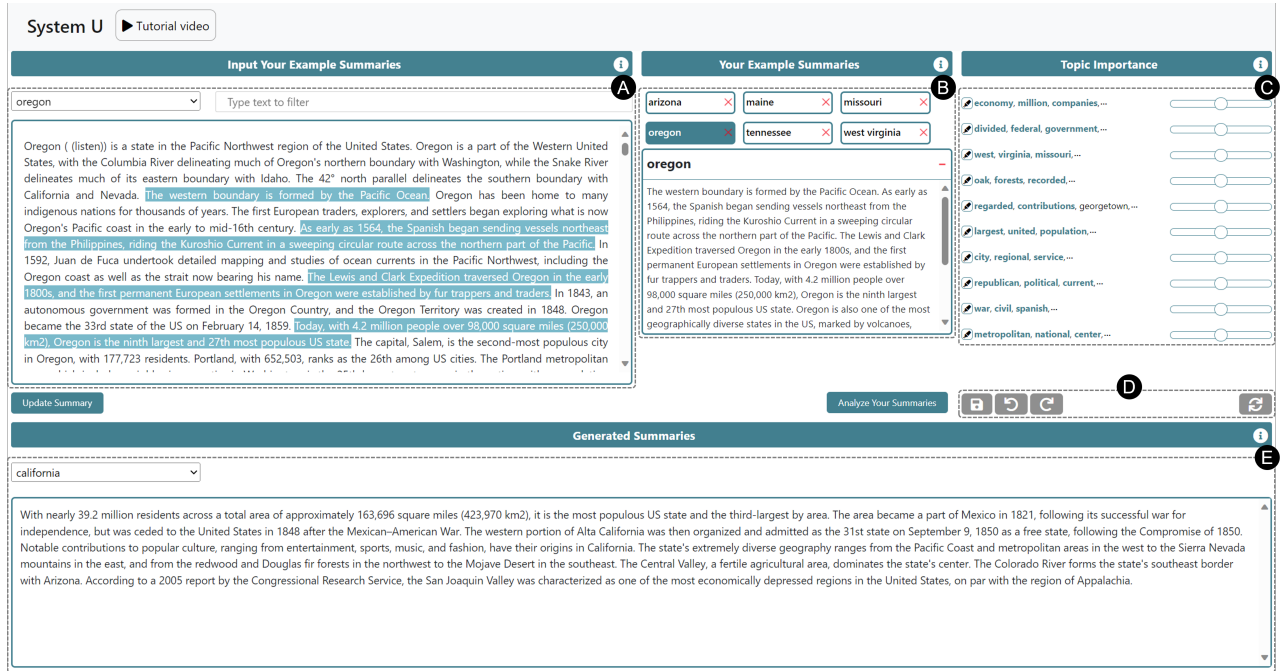
We asked participants to answer the same set of questions after completing their task with each tool: SBERT and Ex2BUNDLE. The questions were designed to evaluate the participants' experience with each tool, including their interactions, the quality of the extracted snippets, and their preferences for future use.

**Post-Task Questionnaire (SBERT).** After completing their task with SBERT, participants answered questions about their experience. Out of the 20 participants with valid responses, one participant didn't submit their responses for the post-task questionnaire for SBERT, so we analyzed the responses from 19 participants. When asked about modifying their example input snippets, 8 modified their example input snippets during the task while 11 did not. Among all participants, 11/20 rated modifying examples as useful or very useful, 5/20 were neutral, and the remaining 3/20 found it somewhat useful or not useful. Regarding ease of modification, 10/20 rated it as easy or very easy, while 5/20 were neutral. For the quality of the extracted snippets, 14/20 participants rated them as good or excellent (9 good, 5 excellent), while 3/20 rated them average and 2/20 rated them poor. When asked whether the extracted snippets helped them make a decision, 14/20 said yes. Only 4/20 reported encountering unexpected information in the

<sup>9</sup>The codes, and themes are available at <https://github.com/kuangfei-long/ex2bundle/tree/main/demo>.



**Figure A1: User study protocol.** Participants first completed a consent form and watched a tutorial video on how to use the assigned tool. Then, they first performed a pre-study questionnaire, followed by a decision-making task using either SBERT or Ex2BUNDLE. After completing the first task with following post-study questionnaire, they performed the same task with the other tool. Finally, they completed a post-study questionnaire comparing both tools and providing feedback on their experience.



**Figure A2: A snapshot of Ex2BUNDLE’s interface.** (A) Users can input examples by clicking on the interface, and search for keywords that they are interested in the documents. (B) Users can review their examples and modify them. (C) On the right side, users can adjust the importance of different topics to tune the retrieved snippets using sliders. (D) Users can save their retrieved snippets. (E) The retrieved snippets are displayed under the examples, the figure showing the snippets in California state.

snippets, one participant stating that the snippets showed some information that is not relevant to their input snippets. Regarding future use, 13/20 agreed or strongly agreed that they would use SBERT to search information across multiple documents, while 2/20 disagreed. Participants who liked SBERT highlighted its ease of use and ability to filter relevant content. Common complaints included slow performance, occasional browser crashes, and the absence of importance sliders for tuning results.

**Post-Task Questionnaire (Ex2BUNDLE).** After completing their task with Ex2BUNDLE, participants answered questions about their experience. We asked them the same questions as in the post-task questionnaire for SBERT, but we also asked them additional questions about their experience with the topic importance sliders, which is a distinctive feature of Ex2BUNDLE. Of the 20 participants with valid responses, 10 modified their example input snippets

while 10 did not. Among all participants, 8/20 found modifying examples useful or very useful, 9/20 were neutral, and 3/20 found it not useful. Regarding ease of modification, 11/20 rated it as easy or very easy, while 7/20 were neutral and 2/20 found it not easy.

A distinctive feature of Ex2BUNDLE is its relative topic importance sliders. The majority of participants (18/20) used the sliders during the task, while 2/20 did not, citing technical issues. Of those who used the sliders, 7/18 observed a change in the extracted snippet after adjusting them. Regarding usefulness of the sliders, 10/20 rated them as useful or very useful, 6/20 were neutral, and 4/20 found them not useful. In terms of ease of use, 15/20 rated the sliders as easy or very easy.

For the quality of the extracted snippets, 12/20 participants rated them as good or excellent (9 good, 3 excellent), while 6/20 average and 2/20 poor or very poor. When asked whether the snippets



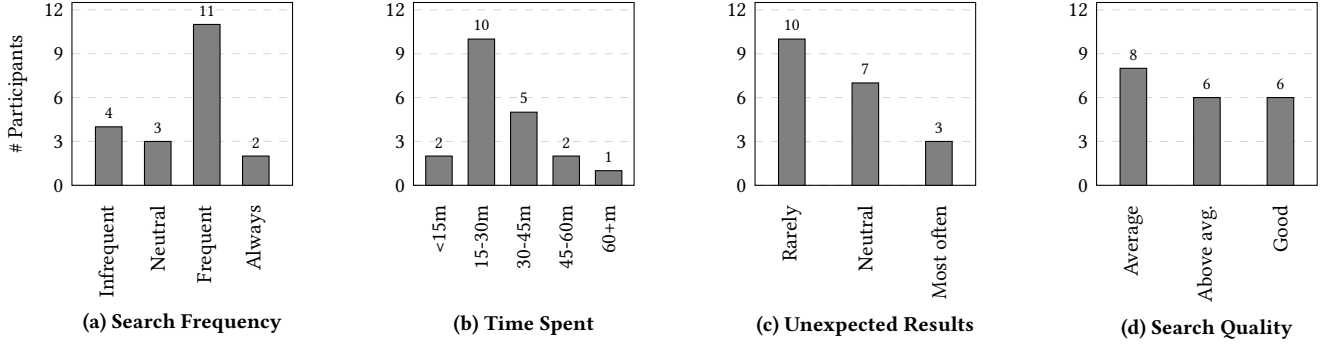


Figure A3: Pre-study questionnaire results (N=20): (a) search frequency from documents or websites, (b) time spent per session (min), (c) frequency of encountering unexpected results, and (d) self-rated search quality.

| Question and Response                                                                                                                  | Count |
|----------------------------------------------------------------------------------------------------------------------------------------|-------|
| <i>What was your overall experience of searching for information using summaries?</i>                                                  |       |
| Excellent                                                                                                                              | 1     |
| Good                                                                                                                                   | 13    |
| Average                                                                                                                                | 3     |
| Very poor                                                                                                                              | 3     |
| <i>While creating example input summaries, did you identify any interesting information that you had not previously thought about?</i> |       |
| Yes                                                                                                                                    | 9     |
| No                                                                                                                                     | 11    |

(a)

| Which tool did you prefer in each category?                                                                         | SBERT | Ex2BUNDLE |
|---------------------------------------------------------------------------------------------------------------------|-------|-----------|
| Which tool generated better summaries?                                                                              | 8     | 12        |
| Which tool did you find more useful?                                                                                | 9     | 11        |
| Which tool did you find easier to use?                                                                              | 4     | 16        |
| Which tool helped you better in making a decision?                                                                  | 8     | 12        |
| Which tool did you find more flexible?                                                                              | 8     | 12        |
| Which tool did you find more comfortable to use?                                                                    | 5     | 15        |
| If you had to choose a tool to search for information from multiple documents/websites, which one would you choose? | 9     | 11        |

(b)

Figure A4: User study satisfaction results. (a) Overall experience of the user study and discovery of new information. Most participants had a good experience with the system, and more than half of the participants discovered new information. (b) Tool preference responses. The majority of participants preferred Ex2BUNDLE over SBERT across all categories, showing that Ex2BUNDLE was generally favored for extracting text snippets for focused intent tasks.

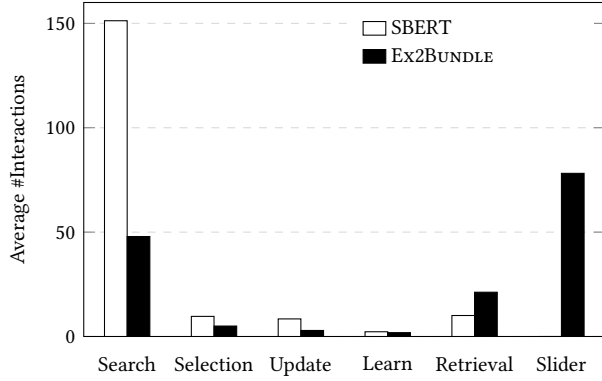
helped them make a decision, 14/20 said yes. For participants who said no, they mentioned that the snippets were not fully covering the information they wanted. Only 3/20 reported finding unexpected information in the snippets, mentioning that some information was not what they expected based on their input snippets. Regarding future use, 11/20 agreed or strongly agreed that they would use Ex2BUNDLE to search information across multiple documents, while 2/20 strongly disagreed, 3/20 disagreed, and 4/20 were neutral. Participants who liked Ex2BUNDLE highlighted the topic importance sliders, easy navigation, and the ability to customize snippet content. Common complaints included slow performance, freezing, and occasional confusion because of the tool’s technical issues.

#### A.4 Quantitative Results after Using Both Tools

We first analyzed the quantitative data by comparing the participants’ ratings such as the summary quality, ease of use, and daily

usage preference for both SBERT and Ex2BUNDLE. We also compared the number of interactions of each type between the two tools to understand how participants engaged with them and how much time they spent working with the tools to support their decision-making process.

**Participants’ satisfaction.** After using both tools, we asked participants to rate the summary quality and ease of use of each tool. In Table A4, overall participants evaluated the experience with retrieving snippets with tools as good (13/20; Table A4 (a)). When comparing SBERT and Ex2BUNDLE, many participants preferred Ex2BUNDLE over SBERT for all dimensions (Table A4 (b)). For example, 12/20 participants preferred Ex2BUNDLE over SBERT for generating better snippets, 11/20 preferred Ex2BUNDLE for usefulness, and 16/20 preferred Ex2BUNDLE for ease of use. When asked which tool they would choose for their usual information-seeking tasks, 11/20 participants preferred Ex2BUNDLE, while 9/20 SBERT.

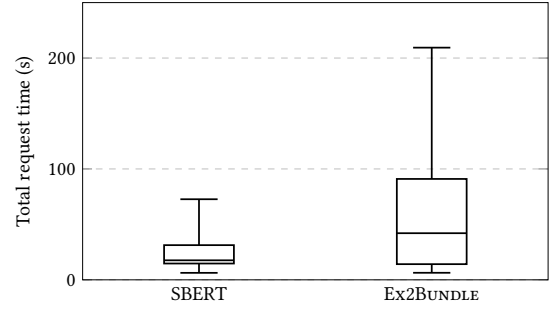


**Figure A5: Comparison between SBERT and Ex2BUNDLE in terms of interaction type counts by averaging the number of interactions per user. Here, *Search*: keyword queries, *Selection*: example selection, *Update*: example modification, *Learn*: intent learning, *Retrieval*: bundle retrieval, *Slider*: slider adjustments.**

**Participants’ interactions.** To understand how participants interacted with the tools, we categorized interactions into six types: *Search*: keyword queries, *Selection*: example selection, *Update*: example modification, *Learn*: intent learning, *Retrieval*: backend retrieval, and *Slider*: interactive parameters. *Search* includes keyword queries that participants used to find relevant information in the Wikipedia articles such as “climate”. *Selection* includes interactions where participants selected specific sentences or sections from the Wikipedia articles to include in the input snippets. *Update* includes interactions where participants modified their selected examples, such as adding or removing sentences from the input snippets. *Learn* includes Ex2BUNDLE’s learning process, where it learns the user’s intent based on their example selections. *Retrieval* includes interactions where participants retrieved snippets with PaQL after providing examples or adjusting sliders. *Slider* includes interactions where participants adjusted the importance sliders to tune the retrieval results. We averaged the count of each interaction type per user and compared the counts between SBERT and Ex2BUNDLE.

Figure A5 shows that SBERT users have more manual interactions: they performed about 3.1 times more *Search* interactions than Ex2BUNDLE users, and selected and updated more examples than Ex2BUNDLE users by about 2.2 and 2.8 times, respectively. However, since Ex2BUNDLE users can adjust the importance sliders unlike SBERT users, they have more *Slider* interactions than SBERT users. Naturally, Ex2BUNDLE users also have about 2.1 times more *Retrieval* interactions than SBERT users, since Ex2BUNDLE retrieves bundles based on slider adjustments, while SBERT does not have such capabilities.

**Participants’ querying time.** We measure the cumulative backend processing time per participant session. This metric reflects backend request processing time rather than end-to-end participant interaction time. We report this metric to demonstrate the level of engagement and interaction participants had with the system. As shown in Figure A6, Ex2BUNDLE has a higher average backend processing time than SBERT, with a median of 42.05 seconds for Ex2BUNDLE compared to 17.61 seconds for SBERT. This suggests that participants interacted more frequently and deeply with



**Figure A6: Average request time (s) for SBERT and Ex2BUNDLE. Ex2BUNDLE has a higher average request time than SBERT, which is expected due to the additional processing steps involved in Ex2BUNDLE such as tuning sliders.**

Ex2BUNDLE, likely due to its interactive features such as the importance sliders and the ability to retrieve snippets based on user input. In contrast, SBERT’s lower processing time may indicate less frequent interactions, as it does not offer the same level of interactivity and customization as Ex2BUNDLE. Furthermore, on average participants retrieved bundles 15.8 times with Ex2BUNDLE, while only 7.8 times with SBERT, which also supports the notion of higher engagement with Ex2BUNDLE.

## A.5 Qualitative Results after Using Both Tools

We analyzed the qualitative data by asking participants to provide feedback on their experience with both tools. We asked them to share their preferences, grievances, and suggestions for improvement. We also asked them to provide feedback on the extracted snippets and how they supported their decision-making process.

**Participants preferred Ex2BUNDLE over SBERT.** Compared to SBERT, our participants preferred Ex2BUNDLE. When comparing SBERT and Ex2BUNDLE, many participants (at least 11/20 for each category; Table A4 (b)) preferred Ex2BUNDLE over SBERT due to its greater customization. P07 mentioned, “*I absolutely loved the sliders. Being able to rank what is most important to me instead of the system trying to guess is great.*” While performing their tasks in SBERT, several participants (5/20) mentioned that it lacked customizability. P16 said, “*There was no way to adjust which factors were more or less important to me. It was tedious to scroll through long Wikipedia articles looking for relevant info to include, and sometimes articles did not address specific things that I wanted to include. I sometimes filtered/searched for certain words or terms, but there were no results, or none relevant.*” However, some participants (4/20) preferred the simplicity that SBERT provides. P20 mentioned, “*It [SBERT] didn’t require much input to generate summaries that captured what I was most interested in, which made it easier to review the information and make a decision.*”

**Topic importance sliders helped to tune snippets.** While working on their tasks using Ex2BUNDLE, the participants had access to a set of topic importance sliders to adjust extracted information. The majority of participants (18/20) used the topic importance sliders to fine-tune their snippets and ensure they reflected the information specific to participants’ needs. P08 explained how they used the sliders to focus on the economy and climate for each state, “*I raised*

the slider up so that the output summaries would prioritize information that pertained to the economy and climate of each state since those were the most important factors for me when it came to choosing a state to work in.” In contrast, P09 used the sliders to reduce the importance of certain topics, “Topics that I wasn’t interested in, I just moved to lower importance. [...] That seems to have put more emphasis on certain topics I cared about [in the summary].” The participants (2/20) who did not use the topic importance sliders stated that their choice was predetermined and that changes to the snippets would not have swayed their decision.

**Extracted snippet supported informed decision-making.** Participants across both conditions (17/20 for SBERT and 20/20 for Ex2BUNDLE) reported that the extracted snippets helped them make informed decisions by enabling them to identify relevant information, gain insight into interesting information, and validate their own preferences. After using Ex2BUNDLE, P04 mentioned, “The summaries show me the details and aspects that I’m interested in while omitting many irrelevant details that I have no interest in knowing. I was able to quickly see which states I find more appealing through reading only the details that I would find relevant.” Similarly, after using SBERT, P07 mentioned, “It was easy to see just the things that were most important to me without the entire Wikipedia page in front of me. Having such detail is nice, but it can be overwhelming. I was able to read the summaries and say, Oh, that doesn’t sound good. I’m going to avoid that, or Awesome! I am going to continue learning more about this state.” Participants were also happy with the quality of the summary and how it organized their preferred data quickly and concisely. P07 pointed out how such efficiencies helped with their tasks, “Being able to switch between the states quickly meant I could see almost instantly how those states compared to each other.”

**Participants’ feedback for improvement.** The participants suggested several improvements. One of the major grievances of participants was the presence of irrelevant information in the extracted snippet. However, P08 found use in such irrelevant snippet while making decisions: “I read information that showed that North Carolina had a high poverty rate, which I did not remember inputting into the input summaries. However, this data helped me to exclude North Carolina from my list of states to live in.” In addition, some noted the documents’ text-heavy nature. They suggested that, while the snippets were useful for identifying relevant information, creating the input snippets was sometimes tedious and often led to confusion, as going through the Wikipedia article to find candidate sentences for the input was time-consuming. Participants also recommended following the document organization and adding an option to filter sections while reading the articles. Additionally, after using Ex2BUNDLE, 3/20 participants suggested that working with the tool has a steep learning curve. P16 mentioned, “It was a bit confusing at first, even after the tutorial, to know exactly how to use it [Ex2BUNDLE] and what to do for the best results. I did not like having to scroll through really long Wikipedia articles for each state in order to find which parts to highlight and use for the summary.”

## APPENDIX B: QUALITY FUNCTION VARIANTS

Figure A7 demonstrates how different quality function variants affect the performance of Ex2BUNDLE in retrieving sentences on the SUBSUME. C-FREQ is the original implementation introduced in

Section 4, which incorporates the term-frequency quality function as the objective function. This variant serves as the base Ex2BUNDLE. **BS** integrates the SBERT cosine-similarity quality function into the Ex2BUNDLE ILP formulation, which maximizes the similarity between selected sentences and the user examples. **P-FREQ** incorporates the TF-IDF and LPR quality functions that capture the importance of sentences based on their frequency and position. **TS** adds a topic-similarity quality function based on cosine similarity, thereby maximizing topic similarity between selected sentences and the examples.

C-FREQ achieves the best performance across all metrics, demonstrating its ability to identify sentences that are semantically aligned with the user examples. Although Ex2BUNDLE relies on this simplified objective function, it outperforms the more complex formulations in terms of both ROUGE scores and semantic similarity. The variant incorporating topic similarity (TS) shows the worst performance across all metrics. This indicates that explicitly maximizing topic similarity is not a reliable signal for selecting sentences that are important and semantically relevant to the examples. Ultimately, while integrating sentence-level semantic similarity or topic similarity seems intuitively promising, they are not effective signals for sentence selection compared to frequency-based objectives. This finding is highly consistent with prior work [41, 73], which observes that frequency-based methods often outperform pure semantic similarity-based methods in retrieval tasks.

### Key Takeaways

- Users can choose different quality functions to optimize for, but the original Ex2BUNDLE formulation with term-frequency quality function (C-FREQ) achieves the best performance across all metrics.
- Frequency-based quality functions outperform semantic similarity-based objectives, suggesting that frequency is a more reliable signal for selecting sentences aligned with user intent in this setting.

## APPENDIX C: DATASET DETAILS

**TPC-H.** For the package query retrieval task, we use the TPC-H benchmark datasets [75], which consists of a set of business-oriented queries and concurrent data modification. For our experiments, we synthesize a `SupplierFeatures` view derived from three tables using a 0.01 scale factor: `Supplier` with 100 tuples, `Partsupp` with 8,000 rows, and `Nation` with 25 rows. Each supplier is represented by five normalized features: *price competitiveness*, *inventory availability*, *financial stability* (account balance), and two geographic indicators (*America* and *Europe*). Ex2BUNDLE infers sum-based bounds for these features by observing three manually provided example packages representing distinct strategies: *Conservative*, *Price-Focused*, and *Balanced*. The optimization objective is to maximize the aggregate utility of cost, availability, and stability:  $Obj = \sum(f_{price} + f_{avail} + f_{bal})$ . We compare Ex2BUNDLE against a *Greedy* baseline, which selects the top- $k$  suppliers by raw objective score while ignoring constraints, and a *Random* baseline.

**SUBSUME** [82]. SUBSUME is a dataset for the evaluation of subjective summary extraction systems. The dataset contains 2,200 (*document*,

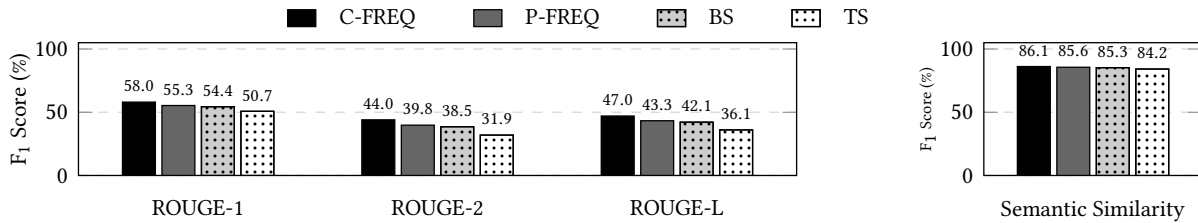


Figure A7: Frequency-based quality functions (C-FREQ & P-FREQ) outperform similarity-based functions (BS & TS) on SUBSUME.

*intent, summary*) triplets over the 50 Wikipedia pages for US states, with ten intents of varying subjectivity, provided by 103 individuals over Mechanical Turk. “Intent” is the underlying question motivating the creation of a summary. In order to evaluate the effectiveness of a *personalized* automatic summarization system, given an intent, we need a few snippets where a subset of the snippets are used as examples provided to the system and the rest are used to evaluate the snippets that the system produces. Accordingly, SUBSUME includes 275 unique (user, intent) pairs, each contributing 8 different, manually curated snippets. During ChatGPT-4o evaluation, five of the snippets are used as example user-inputs for the system to derive a user-intent model in a few-shot manner, and the remaining three are held-back as a test-set to evaluate the system’s ability to capture user-summarization-intent using the five examples. In this work, we explore ten random splits between the example and test sets for each user in the dataset.

**CNN/DailyMail.** The CNN/DailyMail dataset provides snippets in the form of human-written highlights [32]. However, because these highlights are often abstractive they cannot be directly used for extractive tasks. To adapt this dataset for sentence-level extractive summarization, we employed ChatGPT-4o to align the abstractive snippets with their source articles. The model was tasked with identifying the indices of the article sentences that most closely correlate with the reference highlights. To ensure the resulting extractive snippets remained concise and useful, we imposed two structural constraints: the selection could not exceed 10 sentences or 30% of the original article’s sentence count. These constraints prevent the inclusion of excessive or redundant text, forcing the model to prioritize high-density semantic information.

## APPENDIX D: BASELINE DETAILS

**SBERT** [72] is a widely used sentence embedding model that generates semantically meaningful vector representations of sentences. We use SBERT as a simple baseline for example-driven text snippet extraction for selecting sentences that are semantically similar to the example snippets provided by the user based on cosine similarity. We select a snippet using top- $k$  high-scoring sentences in the document, where  $k$  is simply the average number of sentences across the examples.

**BERTSUMEXT** [50] is a widely adopted extractive summarization model. By default, BERTSUMEXT solely considers the input document and extracts important sentences based on general document content, without accommodating specific user inputs. During its training, it learns to identify important sentences by optimizing for word overlap (e.g., ROUGE scores) with gold-standard snippets. To adapt BERTSUMEXT for our example-driven text snippet

extraction task, we first pre-filter the target document, retaining only the candidate sentences that are semantically similar to the user-provided example snippets. We then feed this pre-filtered document into BERTSUMEXT. The intuition behind this approach is that by restricting the input space to sentences semantically aligned with the examples, we effectively bias BERTSUMEXT towards extracting a snippet that reflects the user’s specific intent. In our implementation, we use the BERTSUMEXT model pre-trained on the CNN-DailyMail dataset [32].

**MEMSUM** [27] is a state-of-the-art reinforcement-learning model for long-document extractive summarization. Unlike BERTSUMEXT, MEMSUM is history-aware: it conditions each extraction on the sentences already selected, producing snippets with greater semantic consistency and reduced redundancy. Sentence selection is trained to maximize word overlap with the gold-standard snippet. In this work, we use the model pre-trained on the GovReport dataset [35]. Similarly to BERTSUMEXT, we first pre-filter each target document using the same SBERT-based approach.

**ChatGPT-4o** [59] is a large language model from OpenAI. We use it as a retrieval-augmented baseline for example-driven text snippet extraction to evaluate how well it can understand user intent and learn extraction patterns from a few examples. We use a file attachment method to provide the model with the target document and example snippets, and we prompt the model to generate a snippet that captures the intent expressed in the examples.

## APPENDIX E: PROMPTING DETAILS

*SubSumE.* For the initial set of ChatGPT-4o few-shot experiments, we conducted inference on each user input file, where each file corresponds to a single intent question. The process involved uploading the relevant Wikipedia text files for the states alongside a structured prompt to instruct the model. For instance, one of the prompts used is as follows:

### Few-Shot Prompt for SubSumE

I am providing you with eight text documents. For three of them, Delaware, New Jersey, and Virginia, I will give you example summaries. From these, you need to understand my summarization intent and learn the summarization pattern from the given examples. Then summarize the remaining five documents based on the learned pattern. Make sure to use extractive summarization: select only the original sentences from the given text without generalizing. You can perform detailed summarization by selecting relevant sentences from the text files. Return the final summaries as a combined paragraph.

| Intent in SUBSUME                               | ChatGPT-4o Predicted Intent                                                              |
|-------------------------------------------------|------------------------------------------------------------------------------------------|
| How is the government structured in this state? | What is the governmental and legislative structure of {state}?                           |
| How is the weather of the state?                | What is the climate like in {state}?                                                     |
| What drives the economy in this state?          | What are the economic drivers and geographical characteristics of different U.S. states? |

**Table A8: Comparison of original intents in the SUBSUME dataset and intents predicted by ChatGPT-4o using file attachment for context.**

*CNN/DailyMail*. For the CNN/DailyMail dataset, we used a similar few-shot prompting approach with ChatGPT-4o as SUBSUME. The prompt was designed to guide the model in learning snippet extraction patterns from a few examples and applying them to new articles. The prompt structure is as follows:

#### Few-Shot Prompt for CNN/DailyMail

##### System Instructions:

You are an advanced summarization assistant. I will provide some examples of original articles with its summarization. Please try to learn the pattern of summarization and use it to generate a similar summarization on new given content.

##### Few-Shot Examples:

{{example\_article}}

##### User Input:

The given new article is {{test\_article}}. Please return a list of zero-based sentence indices from the original text that best align with the intent of the summary. Like [1,2].

##### Output Format:

[index\_0, index\_1, ...]

## APPENDIX F: CHATGPT-4O ON SUBSUME

Here we first report the performance of ChatGPT-4o for predicting intents given example snippets. In Table A8, we show the original intents in the SUBSUME dataset and the intents predicted by ChatGPT-4o via an additional query such as “Given these example summaries, what is the summarization intent?”. The original intents are more specific and focused on particular aspects of the states, while the ChatGPT-4o predicted intents tend to be broader or more general. For example, the original intent of “How is the government structured in this state?” is focused on governmental structure, whereas the ChatGPT-4o predicted “What is the governmental and legislative structure of {state}?”, which is more general and encompasses both governmental and legislative aspects.

Table A9 shows the semantic similarity for ChatGPT-4o across focused text snippet extraction tasks, per test state. For Task 1 (“What about this state’s arts and culture attracts you the most?”), the average semantic similarity across the five test states is 0.639, with Utah having the highest similarity of 0.742 and Virginia having the lowest similarity of 0.560. For Task 2 (“What are some of the most interesting things about this state?”), the average semantic similarity across the five test states is 0.649, with California having the highest similarity of 0.737 and Kansas having the lowest

| Example States                                                                 | Test State     | Semantic Similarity |
|--------------------------------------------------------------------------------|----------------|---------------------|
| <b>Task 1: What about this state’s arts and culture attracts you the most?</b> |                |                     |
|                                                                                | Colorado       | 0.662               |
| Delaware,                                                                      | New Jersey     | 0.633               |
| Idaho,                                                                         | Utah           | 0.742               |
| Wisconsin                                                                      | Virginia       | 0.560               |
|                                                                                | West Virginia  | 0.598               |
|                                                                                | <b>Average</b> | <b>0.639</b>        |
| <b>Task 2: What are some of the most interesting things about this state?</b>  |                |                     |
|                                                                                | Arkansas       | 0.693               |
| New Hampshire,                                                                 | California     | 0.737               |
| Hawaii,                                                                        | Georgia        | 0.694               |
| Washington                                                                     | Kansas         | 0.533               |
|                                                                                | Michigan       | 0.587               |
|                                                                                | <b>Average</b> | <b>0.649</b>        |

**Table A9: Semantic similarity for ChatGPT-4o across focused intent text snippet extraction tasks, per test state.**

|                            |                  | EX2BUNDLE | ChatGPT-4o (prompt) | ChatGPT-4o (few-shot) |
|----------------------------|------------------|-----------|---------------------|-----------------------|
| ROUGE-1                    | <b>Precision</b> | 0.2714    | 0.6699              | 0.6655                |
|                            | <b>Recall</b>    | 0.4769    | 0.5155              | 0.5108                |
|                            | <b>F1</b>        | 0.3235    | 0.5504              | 0.5483                |
|                            | <b>F2</b>        | 0.2022    | 0.3440              | 0.3427                |
| ROUGE-2                    | <b>Precision</b> | 0.1364    | 0.5389              | 0.5304                |
|                            | <b>Recall</b>    | 0.2318    | 0.4106              | 0.4021                |
|                            | <b>F1</b>        | 0.1590    | 0.4403              | 0.4332                |
|                            | <b>F2</b>        | 0.0994    | 0.2752              | 0.2708                |
| ROUGE-L                    | <b>Precision</b> | 0.1977    | 0.5873              | 0.5769                |
|                            | <b>Recall</b>    | 0.3445    | 0.4504              | 0.4405                |
|                            | <b>F1</b>        | 0.2335    | 0.4814              | 0.4734                |
|                            | <b>F2</b>        | 0.1459    | 0.3009              | 0.2958                |
| <b>Semantic Similarity</b> |                  | 0.6272    | 0.7965              | 0.7853                |

**Table A10: Focused text snippet extraction on EX2BUNDLE, ChatGPT-4o (prompt), and ChatGPT-4o (few-shot) with manually selected ground truth snippet on CNN/DailyMail dataset crime articles.**

similarity of 0.533. The results give us insights how ChatGPT-4o’s understanding of user intent generally varies across different states, which may be influenced by the specific content and characteristics of each state document.

## APPENDIX G: CNN/DAILYMAIL DETAILS

Here, we provide the detailed results for the generic text snippet extraction and customized focused text snippet extraction tasks on the CNN/DailyMail dataset, as well as the semantic similarity scores for each focused intent text snippet extraction task. For ChatGPT-4o, we report the results for both prompt-only and few-shot prompting approaches. The prompt-only approach uses a single prompt to guide the model’s generation, while the few-shot prompting approach includes three examples for each intent, which are randomly selected from the training set of the CNN/DailyMail dataset.

Table A10 shows that ChatGPT-4o (both prompt and few-shot) significantly outperforms EX2BUNDLE across all ROUGE metrics and semantic similarity, demonstrating its general capability in understanding user intent and generating snippets for generic text



|                     |           | “Crime methods” |                          | “Suspects & Victims” |                          | “Crime locations” |                          |
|---------------------|-----------|-----------------|--------------------------|----------------------|--------------------------|-------------------|--------------------------|
|                     |           | Ex2BUNDLE       | ChatGPT-4o<br>(few-shot) | Ex2BUNDLE            | ChatGPT-4o<br>(few-shot) | Ex2BUNDLE         | ChatGPT-4o<br>(few-shot) |
| ROUGE-1             | Precision | 0.5126          | 0.6831                   | 0.4562               | 0.6901                   | 0.4016            | 0.6825                   |
|                     | Recall    | 0.4489          | 0.6043                   | 0.4215               | 0.5855                   | 0.3783            | 0.4260                   |
|                     | F1        | 0.4608          | 0.6206                   | 0.4207               | 0.5999                   | 0.3686            | 0.5021                   |
|                     | F2        | 0.2880          | 0.3879                   | 0.2629               | 0.3749                   | 0.2304            | 0.3138                   |
| ROUGE-2             | Precision | 0.3009          | 0.5586                   | 0.2493               | 0.5781                   | 0.2110            | 0.5010                   |
|                     | Recall    | 0.2523          | 0.4928                   | 0.2198               | 0.4780                   | 0.1862            | 0.3116                   |
|                     | F1        | 0.2644          | 0.5066                   | 0.2235               | 0.4946                   | 0.1861            | 0.3684                   |
|                     | F2        | 0.1653          | 0.3166                   | 0.1397               | 0.3091                   | 0.1163            | 0.2302                   |
| ROUGE-L             | Precision | 0.3526          | 0.5944                   | 0.3168               | 0.6079                   | 0.2904            | 0.5641                   |
|                     | Recall    | 0.3004          | 0.5256                   | 0.2850               | 0.5088                   | 0.2660            | 0.3498                   |
|                     | F1        | 0.3124          | 0.5397                   | 0.2872               | 0.5242                   | 0.2619            | 0.4135                   |
|                     | F2        | 0.1952          | 0.3373                   | 0.1795               | 0.3276                   | 0.1636            | 0.2584                   |
| Semantic Similarity |           | 0.7435          | 0.8307                   | 0.7021               | 0.8089                   | 0.6117            | 0.7552                   |

Table A11: Focused text snippet extraction task on Ex2BUNDLE and ChatGPT-4o (few-shot) with LLM selected ground truth snippet on crime detailed description.

|                     |           | Generic | “Summarize the crime mentioned in this article.” | “Who are the suspects and victims in this event?” | “Summarize the locations where the crime was committed.” |
|---------------------|-----------|---------|--------------------------------------------------|---------------------------------------------------|----------------------------------------------------------|
| ROUGE-1             | Precision | 0.2714  | 0.5126                                           | 0.4562                                            | 0.4016                                                   |
|                     | Recall    | 0.4769  | 0.4489                                           | 0.4215                                            | 0.3783                                                   |
|                     | F1        | 0.3235  | 0.4608                                           | 0.4207                                            | 0.3686                                                   |
|                     | F2        | 0.2022  | 0.288                                            | 0.2629                                            | 0.2304                                                   |
| ROUGE-2             | Precision | 0.1364  | 0.3009                                           | 0.2493                                            | 0.211                                                    |
|                     | Recall    | 0.2318  | 0.2523                                           | 0.2198                                            | 0.1862                                                   |
|                     | F1        | 0.159   | 0.2644                                           | 0.2235                                            | 0.1861                                                   |
|                     | F2        | 0.0994  | 0.1653                                           | 0.1397                                            | 0.1163                                                   |
| ROUGE-L             | Precision | 0.1977  | 0.3526                                           | 0.3168                                            | 0.2904                                                   |
|                     | Recall    | 0.3445  | 0.3004                                           | 0.285                                             | 0.266                                                    |
|                     | F1        | 0.2335  | 0.3124                                           | 0.2872                                            | 0.2619                                                   |
|                     | F2        | 0.1459  | 0.1952                                           | 0.1795                                            | 0.1636                                                   |
| Semantic Similarity |           | 0.6272  | 0.7435                                           | 0.7021                                            | 0.6117                                                   |

Table A12: Ex2BUNDLE’s performance across various metrics for generic and focused intents for crime category of CNN/DailyMail.

|                     |           | Generic | “Summarize the politicians and celebrities mentioned.” | “Summarize the contributions or impacts of this political event or statement.” | “What are the key issues or debates highlighted in this political event?” |
|---------------------|-----------|---------|--------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| ROUGE-1             | Precision | 0.3068  | 0.4699                                                 | 0.448                                                                          | 0.5055                                                                    |
|                     | Recall    | 0.4029  | 0.397                                                  | 0.5131                                                                         | 0.4703                                                                    |
|                     | F1        | 0.3246  | 0.392                                                  | 0.4626                                                                         | 0.4684                                                                    |
|                     | F2        | 0.2029  | 0.245                                                  | 0.2891                                                                         | 0.2927                                                                    |
| ROUGE-2             | Precision | 0.1395  | 0.2807                                                 | 0.2686                                                                         | 0.2975                                                                    |
|                     | Recall    | 0.1853  | 0.2076                                                 | 0.3019                                                                         | 0.2617                                                                    |
|                     | F1        | 0.1499  | 0.2125                                                 | 0.2751                                                                         | 0.2669                                                                    |
|                     | F2        | 0.0937  | 0.1328                                                 | 0.1719                                                                         | 0.1668                                                                    |
| ROUGE-L             | Precision | 0.2065  | 0.3422                                                 | 0.3126                                                                         | 0.3511                                                                    |
|                     | Recall    | 0.2746  | 0.2675                                                 | 0.36                                                                           | 0.3146                                                                    |
|                     | F1        | 0.2204  | 0.2699                                                 | 0.3233                                                                         | 0.3186                                                                    |
|                     | F2        | 0.1377  | 0.1686                                                 | 0.2021                                                                         | 0.1991                                                                    |
| Semantic Similarity |           | 0.5829  | 0.671                                                  | 0.7454                                                                         | 0.7658                                                                    |

Table A13: Ex2BUNDLE’s performance across various metrics for generic and focused intents for politics category of CNN/DailyMail.

|                     |           | Generic | “Summarize the sports teams or athletes involved.” | “Summarize the match results or the event details.” | “Where is the stadium or location hosting the sports match event.” |
|---------------------|-----------|---------|----------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------|
| ROUGE-1             | Precision | 0.3148  | 0.4544                                             | 0.5007                                              | 0.3875                                                             |
|                     | Recall    | 0.3886  | 0.5049                                             | 0.4372                                              | 0.2503                                                             |
|                     | F1        | 0.3148  | 0.4675                                             | 0.4488                                              | 0.2828                                                             |
|                     | F2        | 0.1967  | 0.2922                                             | 0.2805                                              | 0.1768                                                             |
| ROUGE-2             | Precision | 0.1823  | 0.2903                                             | 0.3285                                              | 0.2407                                                             |
|                     | Recall    | 0.2098  | 0.3095                                             | 0.267                                               | 0.1407                                                             |
|                     | F1        | 0.1743  | 0.2932                                             | 0.2833                                              | 0.1658                                                             |
|                     | F2        | 0.1089  | 0.1833                                             | 0.1771                                              | 0.1036                                                             |
| ROUGE-L             | Precision | 0.2405  | 0.341                                              | 0.3872                                              | 0.3115                                                             |
|                     | Recall    | 0.2852  | 0.3732                                             | 0.3273                                              | 0.1863                                                             |
|                     | F1        | 0.233   | 0.3486                                             | 0.341                                               | 0.2176                                                             |
|                     | F2        | 0.1456  | 0.2179                                             | 0.2131                                              | 0.136                                                              |
| Semantic Similarity |           | 0.6255  | 0.7552                                             | 0.725                                               | 0.6476                                                             |

Table A14: Ex2BUNDLE’s performance across various metrics for generic and focused intents for sports category of CNN/DailyMail.

|                     |           | Generic | “What are the main lifestyle trends or topics highlighted in the article?” | “Summarize the key tips or advice offered for enhancing personal well-being and daily life.” | “What significant insights or stories does the article provide about cultural practices or personal experiences?” |
|---------------------|-----------|---------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| ROUGE-1             | Precision | 0.3056  | 0.5585                                                                     | 0.4078                                                                                       | 0.6109                                                                                                            |
|                     | Recall    | 0.3554  | 0.4031                                                                     | 0.3706                                                                                       | 0.4331                                                                                                            |
|                     | F1        | 0.3136  | 0.4533                                                                     | 0.3638                                                                                       | 0.4931                                                                                                            |
|                     | F2        | 0.196   | 0.2833                                                                     | 0.2274                                                                                       | 0.3082                                                                                                            |
| ROUGE-2             | Precision | 0.124   | 0.3291                                                                     | 0.225                                                                                        | 0.4043                                                                                                            |
|                     | Recall    | 0.1519  | 0.2235                                                                     | 0.1885                                                                                       | 0.2735                                                                                                            |
|                     | F1        | 0.1304  | 0.2596                                                                     | 0.1925                                                                                       | 0.3178                                                                                                            |
|                     | F2        | 0.0815  | 0.1622                                                                     | 0.1203                                                                                       | 0.1986                                                                                                            |
| ROUGE-L             | Precision | 0.1919  | 0.3868                                                                     | 0.2798                                                                                       | 0.4454                                                                                                            |
|                     | Recall    | 0.2268  | 0.2726                                                                     | 0.245                                                                                        | 0.3032                                                                                                            |
|                     | F1        | 0.1982  | 0.3105                                                                     | 0.2445                                                                                       | 0.3514                                                                                                            |
|                     | F2        | 0.1239  | 0.1941                                                                     | 0.1528                                                                                       | 0.2196                                                                                                            |
| Semantic Similarity |           | 0.6016  | 0.7495                                                                     | 0.673                                                                                        | 0.7832                                                                                                            |

Table A15: Ex2BUNDLE’s performance across various metrics for generic and focused intents for lifestyle category of CNN/DailyMail.

snippet extraction tasks. Interestingly, the prompt-only approach performs slightly better than the few-shot approach in terms of ROUGE scores and semantic similarity, which may suggest that providing multiple examples for few-shot prompting does not necessarily lead to better performance in this case.

Table A11 reports the detailed results for the focused text snippet extraction tasks on the CNN/DailyMail dataset contrasting Ex2BUNDLE and ChatGPT-4o (few-shot). Crime articles from CNN/DailyMail contain multi-aspect descriptions (e.g., methods, participants, and locations). However, they are generally written for broad intent. We compare Ex2BUNDLE on this generic intent against ChatGPT-4o (few-shot). We observe that ChatGPT-4o (few-shot) generally outperforms Ex2BUNDLE across all ROUGE metrics and semantic similarity for the focused intent text snippet extraction tasks, suggesting that ChatGPT-4o is more effective at retrieving snippets aligned with broad user intents than Ex2BUNDLE. However, Ex2BUNDLE still shows competitive performance relative to its own generic-intent baseline in Table A10, suggesting that Ex2BUNDLE remains effective at retrieving snippets for specific intents, even though those intents are still relatively broad.

Tables A12 to A15 report the detailed results for the focused text snippet extraction tasks on the CNN/DailyMail dataset with our manually crafted focused intents for crime, politics, sports, and lifestyle categories. Across all categories, the results show that Ex2BUNDLE’s performance is generally better for the focused intent text snippet extraction tasks compared to the generic snippet extraction task. These findings strengthen the evidence that Ex2BUNDLE is effective in retrieving snippets for specific user intents.